大数据，成就未来

Pandas统计分析

2026/10/15

Pandas简介

Pandas是Python下最强大的数据分析和探索工具

- Pandas名称来自于面板（Panel Data）和Python数据分析（Data Analysis）
- 最初被作为金融数据分析工具而开发出来的
- 由AQR Capital Management公司于2008年4月开发出来，2009年底开源
- Pandas功能强大，支持类似于SQL的数据增、删、改、查，并带有丰富的数据处理函数
- 支持时间序列分析功能
- 支持灵活处理缺失数据

雨课堂 Rain Classroom

目录

1
Pandas的数据结构介绍
2
读写不同数据源的数据
3
描述分析DataFrame数据
4
转换与处理时间序列数据
5
使用分组聚合进行组内计算
6
创建透视表与交叉表

Pandas的数据结构介绍

- 要使用pandas，你首先就得熟悉它的两个主要数据结构：
  - Series
  - DataFrame
- 虽然它们并不能解决所有问题，但它们为大多数应用提供了一种可靠的、易于使用的基础。

Pandas的数据结构介绍

Series

一个Series对象，Sobj
Name: population (Sobj.name = 'population')
dtype: float64

state	
California	NaN
Ohio	35000.0
Oregon	16000.0
Texas	71000.0

- 右侧叫Series对象的值(value)
- Sobj.value是一个array
  - array([Nan 35000.0 16000.0 71000.0])

- 左侧叫Series对象的索引(index)
- 索引也可以有名字
  - Sobj.index.name = 'state'

- Series最重要的一个功能是，两个Series对象进行运算时，是根据索引标签自动对齐数据的。

Pandas的数据结构介绍

Series——创建

- Series是一种类似于一维数组的对象，它由一组数据（各种NumPy数据类型）以及一组与之相关的数据标签（即索引）组成。
- 仅由一组数据即可产生最简单的Series：

In [1]:	**obj = pd.Series([4, 7, -5, 3])**
In [2]:	obj
Out[2]:	0 4 1 7 2 -5 3 3 dtype: int64

Series的字符串表现形式为：

- 索引在左边，值在右边。
- 由于我们没有为数据指定索引，于是会自动创建一个0到N-1（N为数据的长度）的整数型索引。

Pandas的数据结构介绍

Series——属性

- 可以通过Series 的**values和index属性**获取其数组表示形式和索引对象：

In [1]:	obj = pd.Series([4, 7, -5, 3])
In [2]:	obj
Out[2]:	0 4 1 7 2 -5 3 3 dtype: int64

In [3]:	obj.values
Out[3]:	array([4, 7, -5, 3])
In [4]:	obj.index
Out[4]:	RangeIndex(start=0, stop=4, step=1)

Pandas的数据结构介绍

Series

- Series对象本身及其索引都有一个**name属性**，该属性跟pandas其他的关键功能关系非常密切：

In [1]:	**obj4.name = 'population'**
In [2]:	**obj4.index.name = 'state'**
In [3]:	obj4
Out[3]:	state California NaN Ohio 35000.0 Oregon 16000.0 Texas 71000.0 Name: population, dtype: float64

obj4
California NaN
Ohio 35000.0
Oregon 16000.0
Texas 71000.0
dtype: float64

Pandas的数据结构介绍

Series——创建

- 通常，我们希望所创建的Series带有一个可以对各个数据点进行标记的索引：

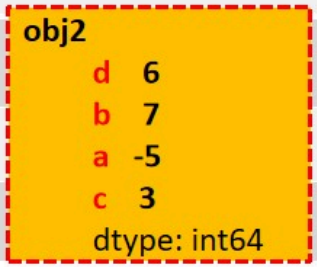
In [1]:	**obj2 = pd.Series([4, 7, -5, 3], index=['d', 'b', 'a', 'c'])**
In [2]:	obj2
Out[2]:	d 4 b 7 a -5 c 3 dtype: int64
In [3]:	**obj2.index**
Out[3]:	Index(['d', 'b', 'a', 'c'], dtype='object')

Pandas的数据结构介绍

Series

- 还可以将Series看成是一个定长的有序字典，因为它是索引值到数据值的一个映射。
- 它可以用在许多原本需要字典参数的函数中：

In [1]:	**'b' in obj2**
Out[1]:	True
In [2]:	**'e' in obj2**
Out[2]:	False



Pandas的数据结构介绍

Series——创建

- 如果数据被存放在一个Python字典中，也可以直接通过这个字典来创建Series：

In [1]:	**sdata = {'Ohio': 35000, 'Texas': 71000, 'Oregon': 16000, 'Utah': 5000}**
In [3]:	**obj3 = pd.Series(sdata)**
In [3]:	obj3
Out[3]:	Ohio 35000 Oregon 16000 Texas 71000 Utah 5000 dtype: int64

Pandas的数据结构介绍

Series

- pandas的isnull和notnull函数可用于检测缺失数据：

In [1]:	pd.isnull(obj4)
Out[1]:	California True Ohio False Oregon False Texas False dtype: bool
In [2]:	**pd.notnull(obj4)**
Out[2]:	California False Ohio True Oregon True Texas True dtype: bool

- Series也有类似的实例方法：

In [3]:	obj4.isnull()
Out[3]:	California True Ohio False Oregon False Texas False dtype: bool

```
obj4
California     NaN
Ohio       35000.0
Oregon     16000.0
Texas      71000.0
dtype: float64
```

Pandas的数据结构介绍

Series

- 对于许多应用而言，Series最重要的一个功能是，它会根据运算的索引标签**自动对齐**数据：

In [1]:	obj3
Out[1]:	Ohio 35000 Oregon 16000 Texas 71000 Utah 5000 dtype: int64
In [2]:	**obj4**
Out[2]:	California NaN Ohio 35000.0 Oregon 16000.0 Texas 71000.0 dtype: float64

In [3]:	obj3 + obj4
Out[3]:	California NaN Ohio 70000.0 Oregon 32000.0 Texas 142000.0 Utah NaN dtype: float64

13

Pandas的数据结构介绍

- 要使用pandas，你首先就得熟悉它的两个主要数据结构：
  - Series
  - **DataFrame**
- 虽然它们并不能解决所有问题，但它们为大多数应用提供了一种可靠的、易于使用的基础。

雨课堂 Rain Classroom

Pandas的数据结构介绍

DataFrame

一个DataFrame对象，Dobj
columns: **['pop', 'state', 'year']**
dtype: float64

	pop	state	year
0	1.5	Ohio	2000
1	1.7	Ohio	2001
2	3.6	Ohio	2002
3	2.4	Nevada	2001
4	2.9	Nevada	2002
5	3.2	Nevada	2003

- 左侧叫DataFrame对象的索引(index)
- **index=[0, 1, 2, 3, 4, 5]**

15

雨课堂 Rain Classroom

Pandas的数据结构介绍

DataFrame——创建

- **创建DataFrame**的办法有很多，最常用的一种是直接传入一个由等长列表或NumPy数组组成的字典：

In [1]:

```
data = {'state': ['Ohio', 'Ohio', 'Ohio', 'Nevada', 'Nevada', 'Nevada'],
        'year': [2000, 2001, 2002, 2001, 2002, 2003],
        'pop': [1.5, 1.7, 3.6, 2.4, 2.9, 3.2]}
```

In [2]:

```
frame = pd.DataFrame(data)
```

In [3]:

```
frame
```

Out[3]:

	pop	state	year
0	1.5	Ohio	2000
1	1.7	Ohio	2001
2	3.6	Ohio	2002
3	2.4	Nevada	2001
4	2.9	Nevada	2002
5	3.2	Nevada	2003

DataFrame会自动加上索引（跟Series一样），且全部列会被有序排列

16

Pandas的数据结构介绍

DataFrame——访问

- 对于特别大的DataFrame，head方法会选取前五行：

In [1]: `frame.head()`

Out[1]:

	pop	state	year
0	1.5	Ohio	2000
1	1.7	Ohio	2001
2	3.6	Ohio	2002
3	2.4	Nevada	2001
4	2.9	Nevada	2002

frame

	pop	state	year
0	1.5	Ohio	2000
1	1.7	Ohio	2001
2	3.6	Ohio	2002
3	2.4	Nevada	2001
4	2.9	Nevada	2002
5	3.2	Nevada	2003

Pandas的数据结构介绍

DataFrame——访问（索引）

- **列可以通过类似字典标记的方式或属性的方式**进行获取，结果为一个Series：

In [1]:	frame2['state']
Out[1]:	one Ohio two Ohio three Ohio four Nevada five Nevada six Nevada Name: state, dtype: object

frame2

	year	state	pop	debt
one	2000	Ohio	1.5	NaN
two	2001	Ohio	1.7	NaN
three	2002	Ohio	3.6	NaN
four	2001	Nevada	2.4	NaN
five	2002	Nevada	2.9	NaN
six	2003	Nevada	3.2	NaN

In [2]:	frame2.year
Out[2]:	one 2000 two 2001 three 2002 four 2001 five 2002 six 2003 Name: year, dtype: int64

注意：

- 返回的Series拥有原DataFrame相同的索引，且其name属性也已经被相应地设置好了。
- 行也可以通过位置或名称的方式进行获取

雨课堂 Rain Classroom

Pandas的数据结构介绍

DataFrame——访问（索引）

- **行可以通过位置或名称**的方式进行获取，比如用loc属性

In [1]:	frame2.loc['three']
Out[1]:	year 2002 state Ohio pop 3.6 debt NaN Name: three, dtype: object

frame2

	year	state	pop	debt
one	2000	Ohio	1.5	NaN
two	2001	Ohio	1.7	NaN
three	2002	Ohio	3.6	NaN
four	2001	Nevada	2.4	NaN
five	2002	Nevada	2.9	NaN
six	2003	Nevada	3.2	NaN

19

Pandas的数据结构介绍

DataFrame——修改

- **列可以通过赋值的方式进行修改**
  - 例如，给那个空的"debt"列赋上一个标量值或一组值：

In [1]:	frame2['debt'] = 16.5
In [2]:	frame2

Out[2]:

	year	state	pop	debt
one	2000	Ohio	1.5	16.5
two	2001	Ohio	1.7	16.5
three	2002	Ohio	3.6	16.5
four	2001	Nevada	2.4	16.5
five	2002	Nevada	2.9	16.5
six	2003	Nevada	3.2	16.5

将一组值给某个列时，其长度必须跟DataFrame的长度相匹配

In [3]:	frame2['debt'] = np.arange(6.)
In [4]:	frame2

Out[4]:

	year	state	pop	debt
one	2000	Ohio	1.5	0.0
two	2001	Ohio	1.7	1.0
three	2002	Ohio	3.6	2.0
four	2001	Nevada	2.4	3.0
five	2002	Nevada	2.9	4.0
six	2003	Nevada	3.2	5.0

Pandas的数据结构介绍

DataFrame——增加新列

- 为不存在的列赋值会创建出一个新列
  - 例如：添加一个新的布尔值的列（state是否为'Ohio'）：

In [1]:
```
frame2['eastern'] = frame2.state == 'Ohio'
```

In [2]:
```
frame2
```

Out[2]:

	year	state	pop	debt	eastern
one	2000	Ohio	1.5	NaN	True
two	2001	Ohio	1.7	-1.2	True
three	2002	Ohio	3.6	NaN	True
four	2001	Nevada	2.4	-1.5	False
five	2002	Nevada	2.9	-1.7	False
six	2003	Nevada	3.2	NaN	False

frame2

	year	state	pop	debt
one	2000	Ohio	1.5	NaN
two	2001	Ohio	1.7	-1.2
three	2002	Ohio	3.6	NaN
four	2001	Nevada	2.4	-1.5
five	2002	Nevada	2.9	-1.7
six	2003	Nevada	3.2	NaN

- 注意：不能用frame2.eastern创建新的列

Pandas的数据结构介绍

DataFrame——删除某列

- del方法可以用来删除某列：

In [1]:	**del frame2['eastern']**
In [2]:	frame2.columns
Out[2]:	Index(['year', 'state', 'pop', 'debt'], dtype='object')

frame2

	year	state	pop	debt
one	2000	Ohio	1.5	NaN
two	2001	Ohio	1.7	-1.2
three	2002	Ohio	3.6	NaN
four	2001	Nevada	2.4	-1.5
five	2002	Nevada	2.9	-1.7
six	2003	Nevada	3.2	NaN

- 注意：通过索引方式返回的列只是相应数据的视图而已，并不是副本。
- 因此，对返回的Series所做的任何就地修改全都会反映到源DataFrame上。
- 通过Series的copy方法即可复制指定列。

Pandas的数据结构介绍

DataFrame——属性

- DataFrame的index和columns也有**name属性**，可以设置：

In [1]:	**frame3.index.name = 'year'; frame3.columns.name = 'state'**
In [2]:	frame3
Out[2]:	state Nevada Ohio year 2000 NaN 1.5 2001 2.4 1.7 2002 2.9 3.6

frame3

year	Nevada	Ohio
2000	NaN	1.5
2001	2.4	1.7
2002	2.9	3.6

Pandas的数据结构介绍

DataFrame——属性

- 跟Series一样，**values属性**也会以二维ndarray的形式返回DataFrame中的数据：

In [1]:	frame3.values
Out[1]:	array([[nan, 1.5], [2.4, 1.7], [2.9, 3.6]])

frame3

state	Nevada	Ohio
year		
2000	NaN	1.5
2001	2.4	1.7
2002	2.9	3.6

雨课堂 Rain Classroom

Pandas的数据结构介绍

DataFrame——属性

- DataFrame.**value数组的dtype一般是object**。因为DataFrame各列的数据类型可能不同，所以value构成的数组内元素就会有不同类型。为了兼容所有列的数据类型，才统一为object（ndarray数组内的所有元素应具有相同的数据类型）

In [1]:	frame3.values
Out[1]:	array([[2000, 'Ohio', 1.5, nan], [2001, 'Ohio', 1.7, -1.2], [2002, 'Ohio', 3.6, nan], [2001, 'Nevada', 2.4, -1.5], [2002, 'Nevada', 2.9, -1.7], [2003, 'Nevada', 3.2, nan]], **dtype=object**)

frame2

	year	state	pop	debt
one	2000	Ohio	1.5	NaN
two	2001	Ohio	1.7	-1.2
three	2002	Ohio	3.6	NaN
four	2001	Nevada	2.4	-1.5
five	2002	Nevada	2.9	-1.7
six	2003	Nevada	3.2	NaN

目录

1
Pandas的数据结构介绍
2
读写不同数据源的数据
3
掌握DataFrame的常用操作
4
描述分析DataFrame数据
5
转换与处理时间序列数据
6
使用分组聚合进行组内计算
7
创建透视表与交叉表

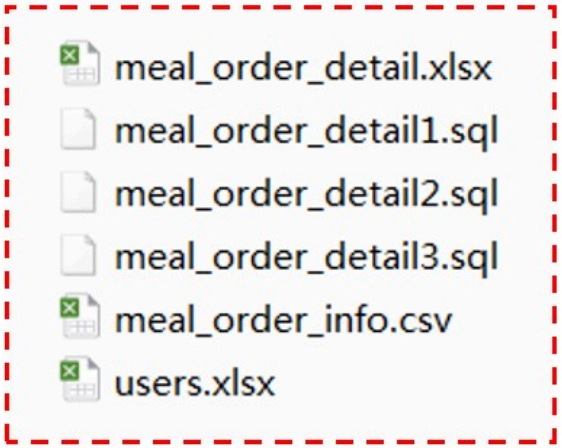
读写不同数据源的数据

- 数据读取是进行数据预处理、建模与分析的前提。不同的数据源，需要使用不同的函数读取。
- pandas 内置了10余种数据源读取函数和对应的数据写人函数。
- 常见的数据源有3种
  - 数据库数据
  - 文本文件(包括一般TXT和CSV文件)
  - Excel文件
- 掌握这3种数据源读取方法，便能够完成80%左右的数据读取工作。

读写不同数据源的数据

数据来源分析

- 以餐饮销售数据为例，该数据存在于3个系统中。
- 3个系统中的数据源并不相同，所以需要使用多种数据读取方式读取相应数据。
  - 订单详情（数据库数据）；
  - 订单信息表（CSV数据）；
  - 客户信息（Excel数据）

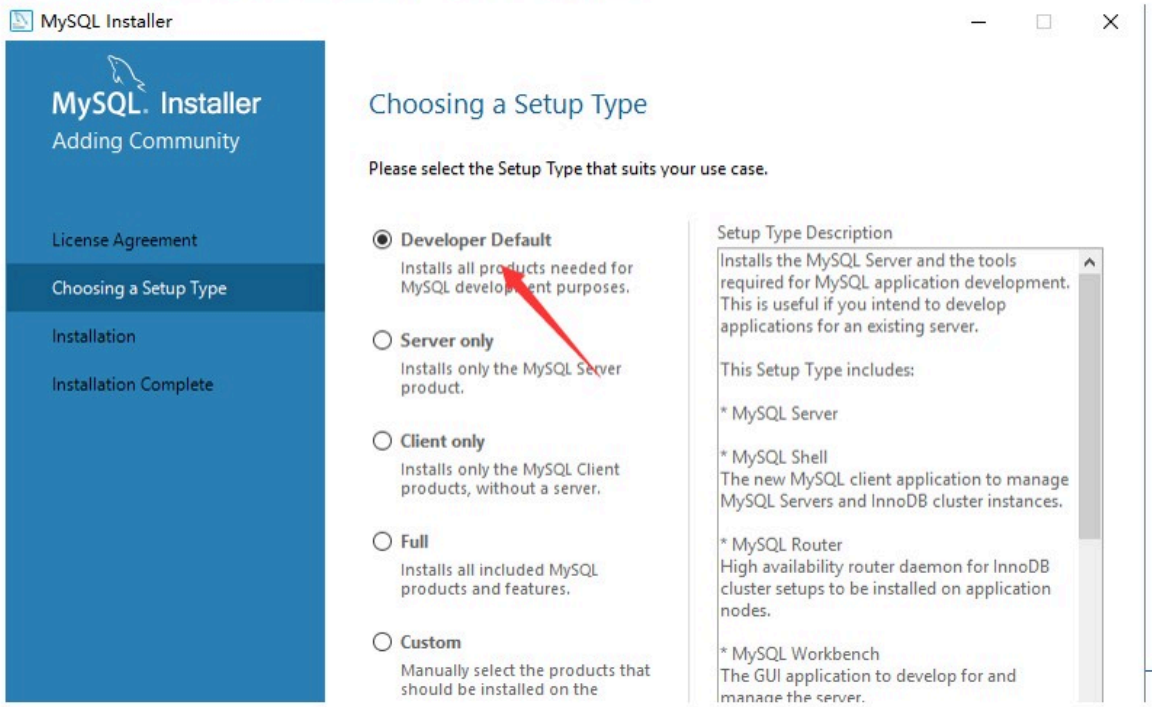


- 在生产环境中，绝大多数的数据都存储在数据库中
- .sql文件是MySQL数据库的备份文件

读写数据库数据

MySQL数据库安装 —— 方法1

- mysql-installer-community-8.0.18.0.msi

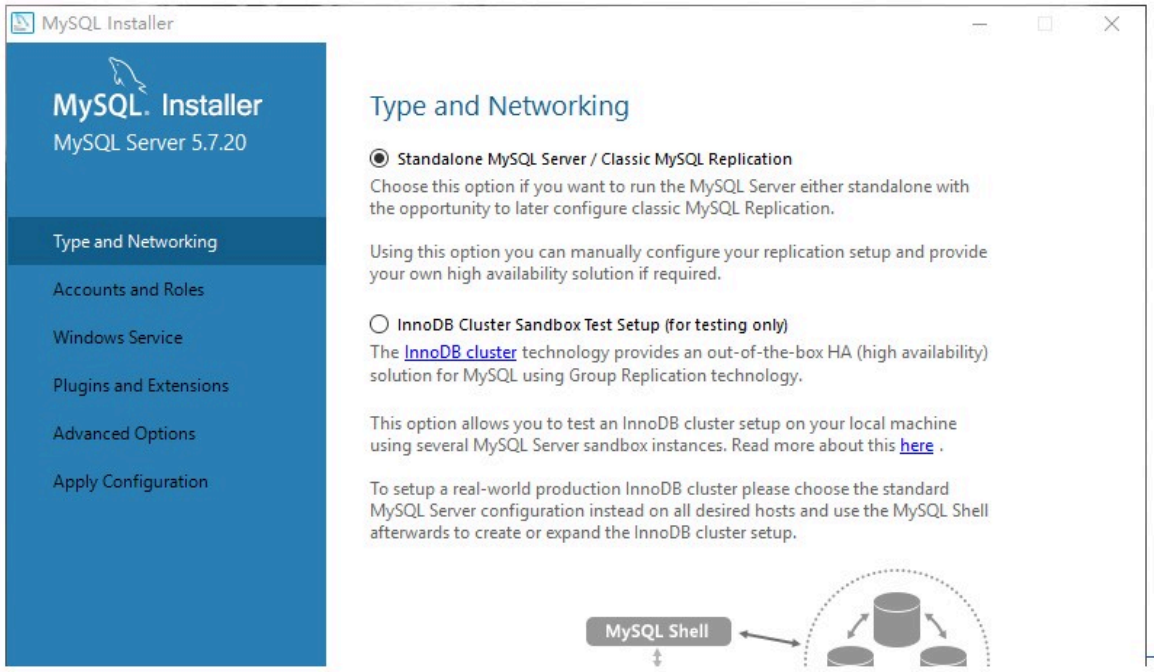


雨课堂 Rain Classroom

读写数据库数据

MySQL数据库安装 —— 方法1

- mysql-installer-community-8.0.18.0.msi

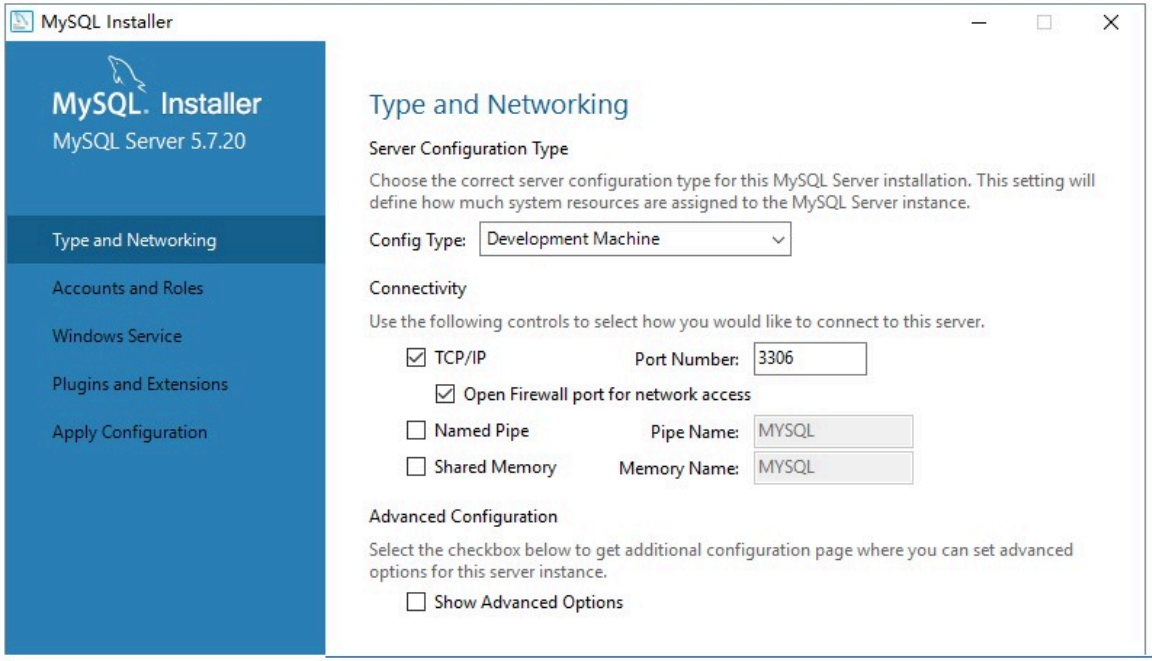


30

读写数据库数据

MySQL数据库安装 —— 方法1

- mysql-installer-community-8.0.18.0.msi

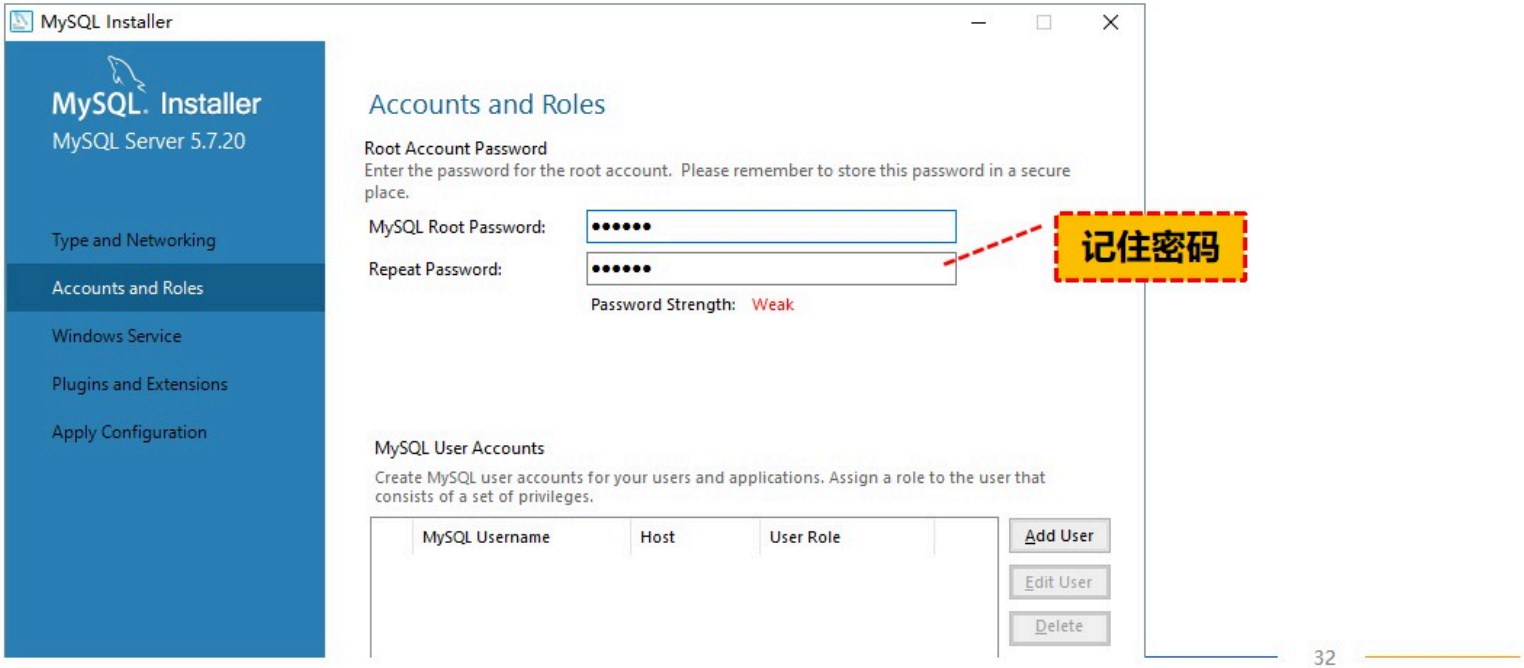


31

读写数据库数据

MySQL数据库安装 —— 方法1

- mysql-installer-community-8.0.18.0.msi

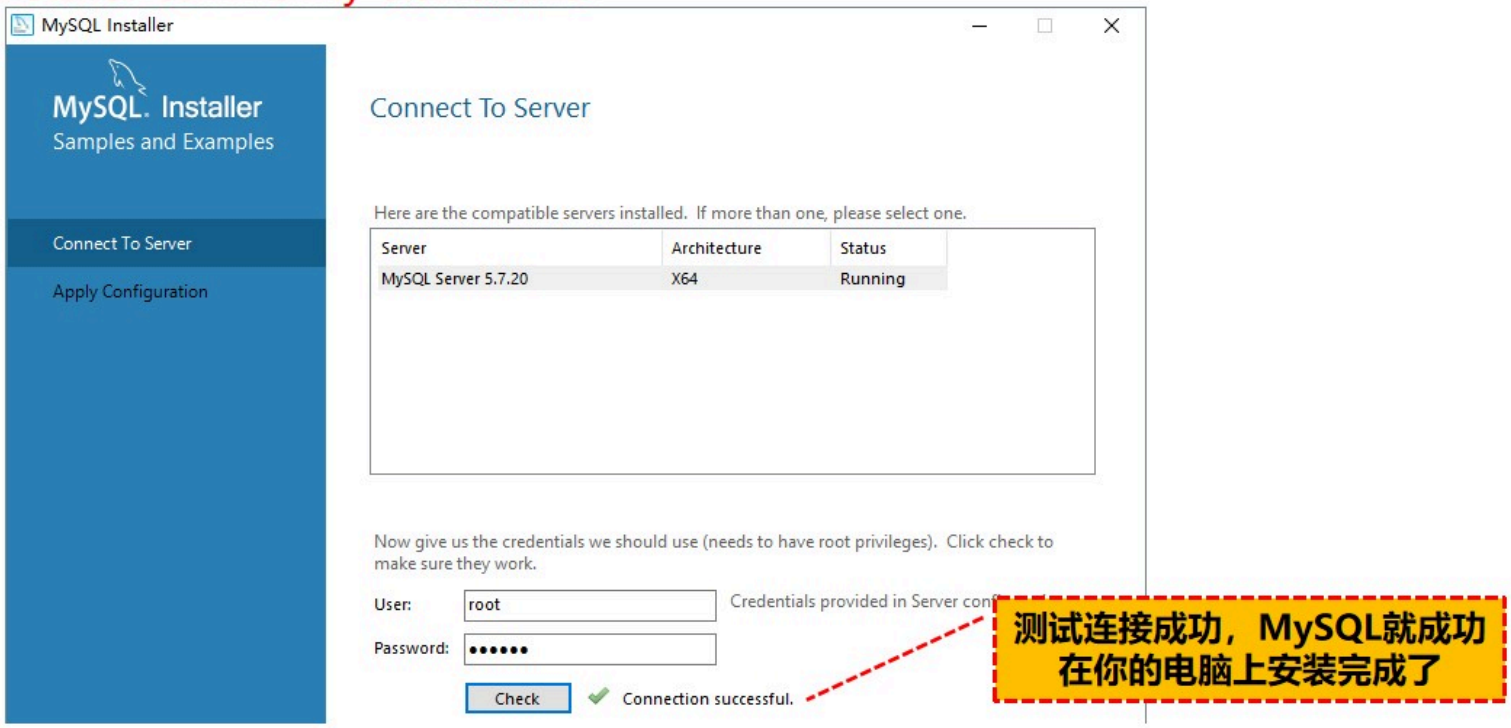


32

读写数据库数据

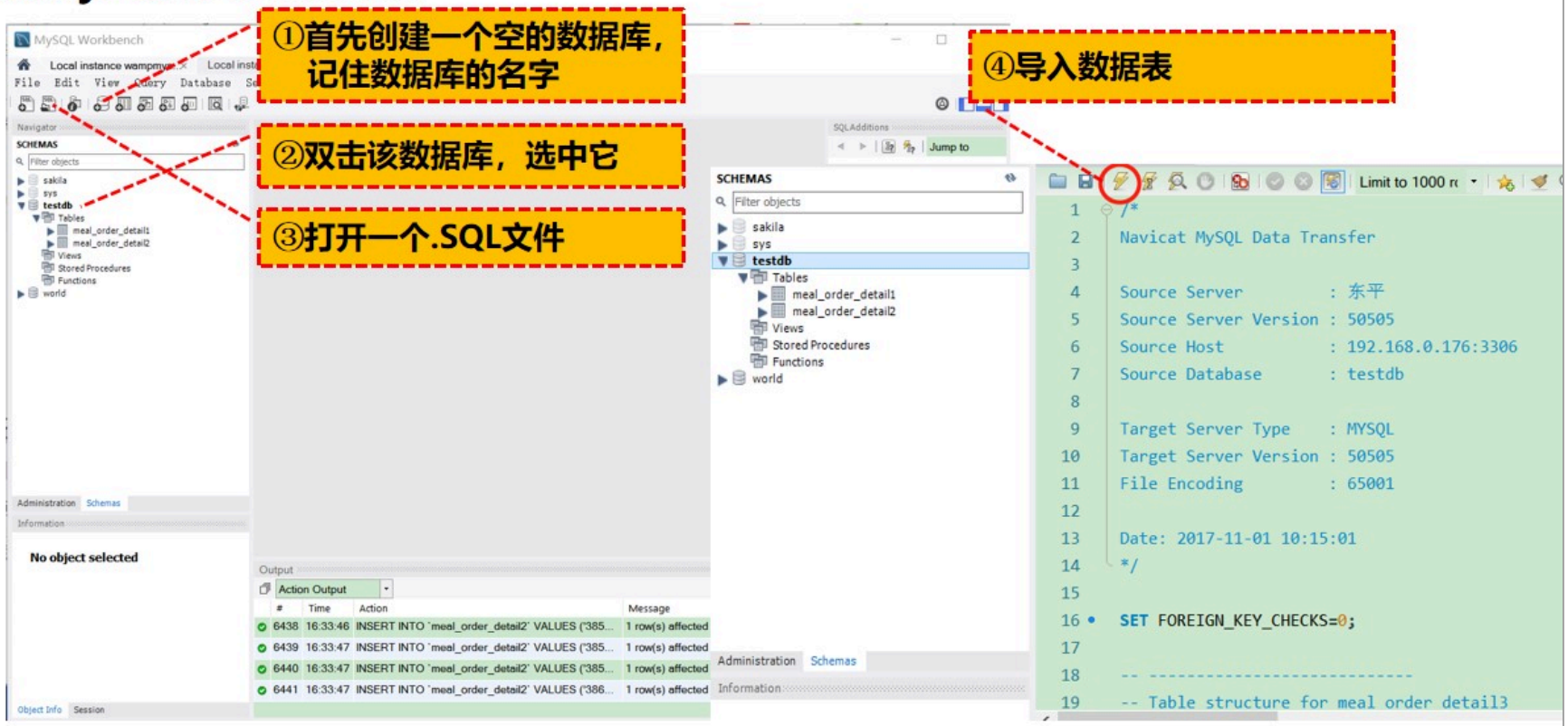
MySQL数据库安装 —— 方法1

- mysql-installer-community-8.0.18.0.msi



读写数据库数据

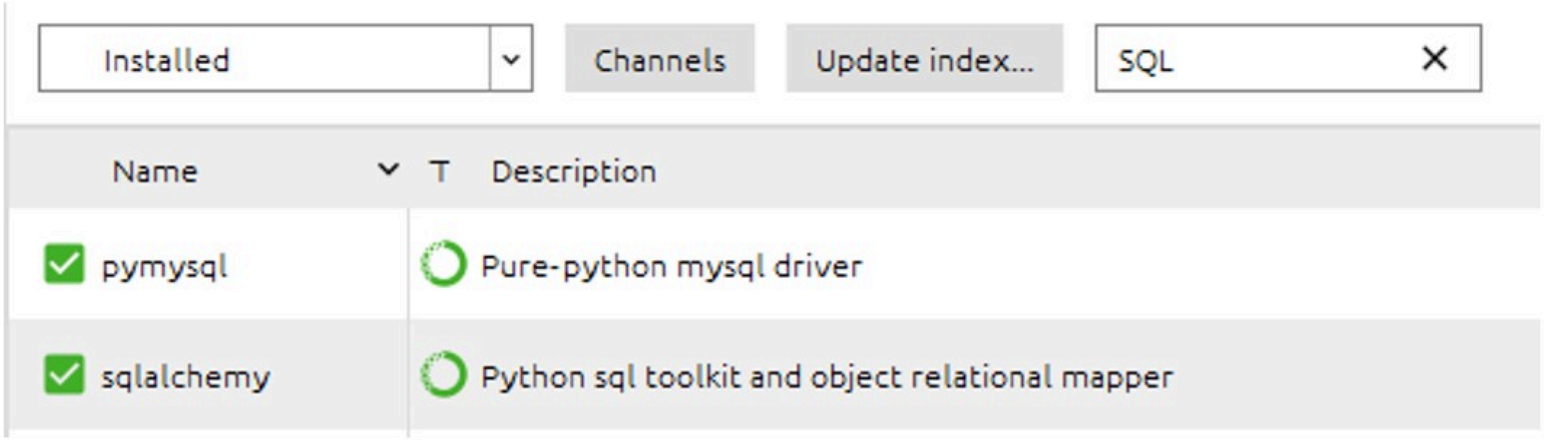
使用MySQLworkbench



读写数据库数据

1.数据库数据读取

- 需要在anaconda中安装这两个库：SQLAlchemy库和pymysql库



- 如果Anaconda中没有，则启动它的命令行方式，进行安装
  - pip install pymysql

35

读写数据库数据

1. 连接数据库

- **使用create_engine函数，建立一个数据库连接**
  - create_engine函数的参数是一个连接字符串
  - 在使用Python的SQLAlchemy时，MySQL数据库连接字符串的格式如下：

数据库产品名+连接工具名://用户名:密码@数据库IP地址:数据库端口号/数据库名称?charset = 数据库数据编码

例如：'**mysql+pymysql://root:1234@127.0.0.1:3306/testdb?charset=utf8**'

雨课堂 Rain Classroom

读写数据库数据

1. 连接数据库 —— 实例

- 使用create_engine函数，建立一个数据库连接

```
from sqlalchemy import create_engine
#创建一个mysql连接器，用户名为root，密码为1234，地址为127.0.0.1，数据库名称为testdb，编码为utf-8
engine =
    create_engine('mysql+pymysql://root:1234@127.0.0.1:3306/testdb?charset=utf8')
print(engine)
```

输出结果：

```
Engine(mysql+pymysql://root:***@127.0.0.1:3306/testdb?charset=utf8)
```

- Python和数据库建立了连接之后，就可以使用pandas对数据库进行读取了。
- pandas实现数据库读取有3个函数：

37

读写数据库数据

2. 数据库数据读取

- read_sql_table只能够读取数据库的某一个表格，不能实现查询的操作。

  pandas.read_sql_table(table_name, con, schema=None, index_col=None, coerce_float=True, columns=None)

- read_sql_query则只能实现查询操作，不能直接读取数据库中的某个表。

  pandas.read_sql_query(sql, con, index_col=None, coerce_float=True)

- read_sql是两者的综合，既能够读取数据库中的某一个表，也能够实现查询操作。

  pandas.read_sql(sql, con, index_col=None, coerce_float=True, columns=None)

38

雨课堂 Rain Classroom

读写数据库数据

2. 数据库数据读取 —— 实例

- **read_sql_table**只能够读取数据库的某一个表格，不能实现查询的操作

表名

```
In [12]: detail1 = pd.read_sql_table('meal_order_detail1',con = engine)

In [13]: print('使用read_sql_table读取订单详情表的长度为:',len(detail1))
使用read_sql_table读取订单详情表的长度为: 2779

In [14]: detail1
Out[14]:
     detail_id order_id dishes_id  ...   bar_code      picture_file emp_id
0         2956      417    610062  ...         NA  caipu/104001.jpg   1442
1         2958      417    609957  ...         NA  caipu/202003.jpg   1442
2         2961      417    609950  ...         NA  caipu/303001.jpg   1442
3         2966      417    610038  ...         NA  caipu/105002.jpg   1442
4         2968      417    610003  ...         NA  caipu/503002.jpg   1442
5         1899      301    610019  ...         NA  caipu/204002.jpg   1095
```


雨课堂 Rain Classroom

读写数据库数据

2. 数据库数据读取 —— 实例

- read_sql_query则只能实现查询操作，不能直接读取数据库中的某个表。

SQL语句，显示数据库中所有表

```
In [15]: formlist = pd.read_sql_query('show tables', con = engine)
    ...: print('testdb数据库数据表清单为:','\n',formlist)
testdb数据库数据表清单为:
      Tables_in_testdb
0  meal_order_detail1
1  meal_order_detail2
2  meal_order_detail3
```

读写数据库数据

2. 数据库数据读取 —— 实例

- **read_sql**是两者的综合，既能够读取数据库中的某一个表，也能够**实现查询操作**。

SQL语句

```
In [24]: detail2 = pd.read_sql('select * from meal_order_detail2 where order_id=366',
    ...:         con = engine)
    ...: print('使用read_sql函数+sql语句读取的订单详情表长度为:',len(detail2))
    ...: print(detail2)
使用read_sql函数+sql语句读取的订单详情表长度为: 7
  detail_id order_id dishes_id  ...   bar_code                picture_file emp_id
0      2352      366    609967  ...         NA           caipu/101002.jpg   1159
1      2354      366    609961  ...         NA           caipu/102001.jpg   1159
2      2356      366    606000  ...         NA           caipu/201001.jpg   1159
3      2358      366    606106  ...         NA  /jsp/pc/images/606106.jpg   1159
4      2361      366    610003  ...         NA           caipu/503002.jpg   1159
5      2360      366    610043  ...         NA           caipu/305002.jpg   1159
6      2362      366    610011  ...         NA           caipu/601005.jpg   1159

[7 rows x 19 columns]
```


雨课堂 Rain Classroom

读写数据库数据

3. 数据库数据存储

数据库表名　数据库链接　不将行索引写入数据库　如表存在则替换原表

```
detail1.to_sql('test1',con = engine,index = False,if_exists = 'replace')
formlist1 = pd.read_sql_query('show tables',con = engine)
print('新增一个表格后testdb数据库数据表清单为：\n', formlist1)
```

```
新增一个表格后testdb数据库数据表清单为：
      Tables_in_testdb
0  meal_order_detail1
1  meal_order_detail2
2  meal_order_detail3
3               test1
```

读写文本文件

1. 文本文件读取

- 使用**read_table**来读取文本文件。

pandas.read_table(filepath_or_buffer, sep='\t', header='infer', names=None, index_col=None, dtype=None, engine=None, nrows=None)

- 使用**read_csv**函数来读取csv文件。

pandas.read_csv(filepath_or_buffer, sep=',', header='infer', names=None, index_col=None, dtype=None, engine=None, nrows=None)

读写文本文件

1. 文本文件读取

- 还可以使用read_table，并指定分隔符：

In [1]:	**pd.read_table('examples/ex1.csv', sep=',')**
In [2]:	df
Out[2]:	a b c d message 0 1 2 3 4 hello 1 5 6 7 8 world 2 9 10 11 12 foo

examples/ex1.csv
a,b,c,d,message
1,2,3,4,hello
5,6,7,8,world
9,10,11,12,foo

读写文本文件

1. 文本文件读取

- 有些情况下，有些文件可能不是用固定的分隔符去分隔字段的（而是空白符或其它模式）。
- 这种情况下，可以传递一个正则表达式作为read_table的分隔符。

```
In [1]: result = pd.read_table('examples/ex3.txt', sep='\s+')
```

```
In [2]: result
```

Out[2]:

	A	B	C
aaa	-0.264438	-1.026059	-0.619500
bbb	0.927272	0.302904	-0.032399
ccc	-0.264273	-0.386314	-0.217601
ddd	-0.871858	-0.348382	1.100491

examples/ex3.txt

	A	B	C
aaa	-0.264438	-1.026059	-0.619500
bbb	0.927272	0.302904	-0.032399
ccc	-0.264273	-0.386314	-0.217601
ddd	-0.871858	-0.348382	1.100491

- 这里，由于列名数量比数据行的列数少1，所以read_table推断第一列应该是DataFrame的索引。

读写文本文件

1. 文本文件读取

- 这些解析器函数还有许多参数可以帮助你处理各种各样的异形文件格式。
- 例如，可以用skiprows跳过文件的第一行、第三行和第四行：

In [1]:

```
pd.read_csv('examples/ex4.csv', skiprows=[0, 2, 3])
```

Out[1]:

	a	b	c	d	message
0	1	2	3	4	hello
1	5	6	7	8	world
2	9	10	11	12	foo

examples/ex4.csv

```
# hey!
a,b,c,d,message
# just wanted to make things more difficult for you
# who reads CSV files with computers, anyway?
1,2,3,4,hello
5,6,7,8,world
9,10,11,12,foo
```


雨课堂 Rain Classroom

读写文本文件

1. 文本文件读取

- 缺失值处理是文件解析任务中的一个重要组成部分。
- 缺失数据经常两种情况：①没有（空字符串）；②用某个标记值表示（NA或NULL）

In [1]:
```
result = pd.read_csv('examples/ex5.csv')
```

In [2]:
```
result
```

Out[2]:

	something	a	b	c	d	message
0	one	1	2	3.0	4	NaN
1	two	5	6	NaN	8	world
2	three	9	10	11.0	12	foo

examples/ex5.csv

```
something,a,b,c,d,message
one,1,2,3,4,NA
two,5,6,,8,world
three,9,10,11,12,foo
```

47

读写文本文件

2. 文本文件储存

- 文本文件的存储和读取类似，结构化数据可以通过Pandas中的to_csv函数实现。

DataFrame.to_csv(path_or_buf=None, sep=',', na_rep='', columns=None, header=True, index=True, index_label=None, mode='w', encoding=None)

雨课堂 Rain Classroom

读写Excel文件

1. Excel文件读取

- Excel是微软公司的办公软件Microsof Office 的组件之一，它可以对数据进行处理、统计分析等操作，广泛地应用于管理、财经和金融等众多领域。其文件保存依照程序版本的不同分为两种。

(1) Microsoft Oflice Excel 2007之前的版本(不包括2007)默认保存的文件扩展名为.xls。

(2) Microsoft Ofice Excel 2007之后的版本默认保存的文件扩展名为.xlsx。

- pandas提供了**read_excel**函数来读取“xls” “xlsx”两种Excel文件。

pandas.read_excel(io, sheetname=0, header=0, index_col=None, names=None, dtype=None)

文件名 | 表格号 | 表头行号 | 行索引 | 定义表头

49

读写Excel文件

1. Excel文件读取 —— 实例

```
user = pd.read_excel('data/users.xlsx')      # 读取user.xlsx文件
print('客户信息表长度为：', len(user))
```

```
客户信息表长度为： 734

In [23]: user
Out[23]:
     USER_ID   MYID ACCOUNT     NAME  ...   sex   poo  address   age
0          1  admin   超级管理员    admin  ...    男  广东广州     泰迪科技  23.0
1        981    NaN      老师  teacher  ...   NaN   NaN      NaN   NaN
2        982    NaN     叶亦凯       sx  ...    男  广东广州      广州  21.0
3        983    NaN     邓彬彬      lyy  ...    女  广东广州      广州  21.0
4        984    NaN     张建涛      zad  ...    男  广东广州      广州  22.0
5        985    NaN     莫诗怡       mf  ...    女    广西      广州  23.0
6        986    NaN     莫子建      mjc  ...    男    广西      广州  22.0
7        987    NaN     易子歆      yll  ...    女    广西      广州  21.0
```


雨课堂 Rain Classroom

读写Excel文件

2. Excel文件储存

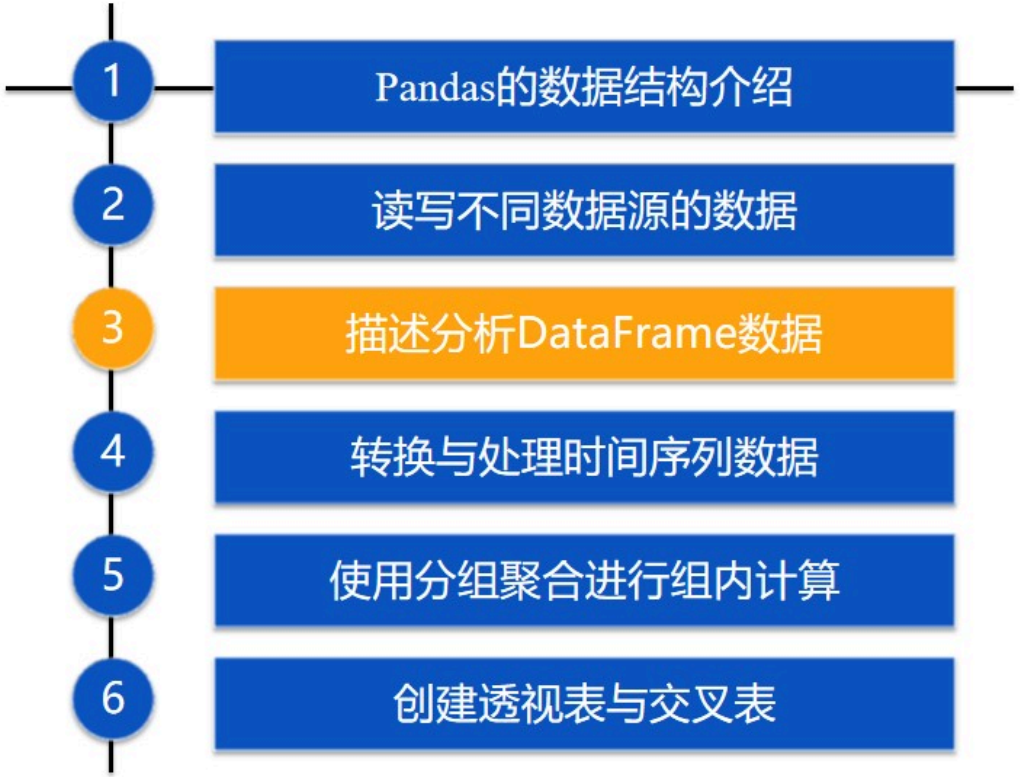
- 将文件存储为Excel文件，可以使用**to_excel**方法。其语法格式如下。

DataFrame.to_excel(excel_writer=None, sheetname=None', na_rep='', header=True, index=True, index_label=None, mode='w', encoding=None)

- 与to_csv方法的常用参数基本一致，区别之处在于指定存储文件的文件路径参数名称为excel_writer，并且没有sep参数，增加了一个sheetnames参数用来指定存储的Excel sheet的名称，默认为sheet1。

雨课堂 Rain Classroom

目录


1
Pandas的数据结构介绍
2
读写不同数据源的数据
3
描述分析DataFrame数据
4
转换与处理时间序列数据
5
使用分组聚合进行组内计算
6
创建透视表与交叉表

了解数据

- DataFrame是最常用的pandas对象，类似于Microsoft Office Excel表格。
- 完成数据读取后，数据就以DataFrame数据结构存储在内存中。
- 但此时并不能直接开始统计分析工作，需要使用DataFrame的属性与方法对数据的分布、大小等基本的数据状况有一个了解。
- 只有对数据基本状况有了一个深度的了解，才能够依据数据的状况，进行量身定制的统计分析。

了解数据

任务分析

(1) 查看数据的大小、维度等基本信息。

(2) 查看销售菜品数据的基本统计信息。

(3) 剔除全为空值或者所有元素取值相同的列。

雨课堂 Rain Classroom

查看DataFrame的常用属性

基础属性

函数	返回值
values	元素
index	行索引
columns	列名
dtypes	类型
size	元素个数
ndim	维度数
shape	数据形状（行列数目）

55

查看DataFrame的常用属性

```
from sqlalchemy import create_engine
import pandas as pd
# 创建数据库连接
engine =
    create_engine('mysql+pymysql://root:1234@127.0.0.1:3306/testdb?charset=utf8')
detail= pd.read_sql_table('meal_order_detail1', con = engine)

print('订单详情表的索引为：', detail.index)
print('订单详情表的所有值为：\n', detail.values)
print('订单详情表的列名为：\n', detail.columns)
print('订单详情表的数据类型为：\n', detail.dtypes)
```

查看DataFrame的常用属性

```
print('订单详情表的索引为：', detail.index)
```

```
订单详情表的索引为： RangeIndex(start=0, stop=2779, step=1)
```

```
print('订单详情表的所有值为：\n', detail.values)
```

```
订单详情表的所有值为：
[['2956' '417' '610062' ... 'NA' 'caipu/104001.jpg' '1442']
['2958' '417' '609957' ... 'NA' 'caipu/202003.jpg' '1442']
['2961' '417' '609950' ... 'NA' 'caipu/303001.jpg' '1442']
...
['6756' '774' '609949' ... 'NA' 'caipu/404005.jpg' '1138']
['6763' '774' '610014' ... 'NA' 'caipu/302003.jpg' '1138']
['6764' '774' '610017' ... 'NA' 'caipu/302006.jpg' '1138']]
```

查看DataFrame的常用属性

```
print('订单详情表的列名为：\n', detail.columns)
```

```
订单详情表的列名为：
 Index(['detail_id', 'order_id', 'dishes_id', 'logicprn_name',
       'parent_class_name', 'dishes_name', 'itemis_add', 'counts', 'amounts',
       'cost', 'place_order_time', 'discount_amt', 'discount_reason',
       'kick_back', 'add_inprice', 'add_info', 'bar_code', 'picture_file',
       'emp_id'],
      dtype='object')
```


雨课堂 Rain Classroom

查看DataFrame的常用属性

```
print('订单详情表的数据类型为：\n', detail.dtypes)
```

```
订单详情表的数据类型为：
 detail_id                object
order_id                  object
dishes_id                 object
logicprn_name             object
parent_class_name         object
dishes_name               object
itemis_add                object
counts                   float64
amounts                  float64
cost                      object
place_order_time    datetime64[ns]
discount_amt              object
discount_reason           object
kick_back                 object
add_inprice               object
add_info                  object
bar_code                  object
picture_file              object
emp_id                    object
dtype: object
```

查看DataFrame的常用属性

```
print('订单详情表的元素个数为：', detail.size)          # 查看DataFrame的元素个数
print('订单详情表的维度数为：', detail.ndim)            # 查看DataFrame的维度数
print('订单详情表的形状为：', detail.shape)             # 查看DataFrame的形状
```

```
订单详情表的元素个数为： 52801
订单详情表的维度数为： 2
订单详情表的形状为： (2779, 19)
```


雨课堂 Rain Classroom

查看DataFrame的常用属性

```
print('订单详情表转置前形状为：',detail.shape)
print('订单详情表转置后形状为：',detail.T.shape)
```

```
订单详情表转置前形状为： (2779, 19)
订单详情表转置后形状为为： (19, 2779)
```

```
In [42]: detail.T
Out[42]:
                              0          ...                    2778
detail_id                  2956          ...                    6764
order_id                    417          ...                     774
dishes_id                610062          ...                  610017
logicprn_name                NA          ...                      NA
parent_class_name            NA          ...                      NA
dishes_name                蒜蓉生蚝          ...                    酸辣藕丁
itemis_add                   0          ...                       0
counts                       1          ...                       1
amounts                     49          ...                      33
cost                         NA          ...                      NA
place_order_time  2016-08-01 11:05:00          ...  2016-08-10 22:04:00
```

61

分析DataFrame数据

1. 数值型特征的描述性统计——NumPy中的描述性统计函数

- 数值型数据的描述性统计主要包括了计算数值型数据的完整情况、最小值、均值、中位数、最大值、四分位数、极差、标准差、方差、协方差和变异系数等。
- 在NumPy库中已经提到了很多统计函数，一些常用的统计学函数如下表所示：

函数名称	说明	函数名称	说明
np.min	最小值	np.max	最大值
np.mean	均值	np.ptp	极差
np.median	中位数	np.std	标准差
np.var	方差	np.cov	协方差

- **pandas库基于NumPy，自然也可以用这些函数对DataFrame进行描述性统计。**

62

雨课堂 Rain Classroom

描述分析DataFrame数据

1. 数值型特征的描述性统计

- NumPy中的描述性统计函数

```
import numpy as np
print('订单详情表中amount（价格）的平均值为：', np.mean(detail['amounts']))
```

```
订单详情表中amount（价格）的平均值为：  45.343084145901045
```

- pandas描述性统计方法

```
print('订单详情表中amount（价格）的平均值为：', detail['amounts'].mean())
```

```
订单详情表中amount（价格）的平均值为：  45.343084145901045
```


雨课堂 Rain Classroom

描述分析DataFrame数据

1. 数值型特征的描述性统计—— pandas描述性统计方法

- pandas提供了与统计相关的主要方法，如下表：

方法名称	说明	方法名称	说明
min	最小值	max	最大值
mean	均值	ptp	极差
median	中位数	std	标准差
var	方差	cov	协方差
sem	标准误差	mode	众数
skew	样本偏度	kurt	样本峰度
quantile	四分位数	count	非空值数目
describe	描述统计	mad	平均绝对离差

描述分析DataFrame数据

1. 数值型特征的描述性统计

- pandas还提供了一个方法叫作**describe**，能够一次性得出DataFrame所有数值型特征的非空值数目、均值、四分位数、标准差。

```
print('订单详情表counts和amounts两列的描述性统计为：\n', detail[['counts','amounts']].describe())
```

```
订单详情表counts和amounts两列的描述性统计为：
         counts       amounts
count   2769.0   2769.000000
mean       2.0     45.343084
std        0.0     36.841316
min        2.0      1.000000
25%        2.0     25.000000
50%        2.0     35.000000
75%        2.0     56.000000
max        2.0    178.000000
```


雨课堂 Rain Classroom

描述分析DataFrame数据

2. 类别型特征的描述性统计方法

- pandas提供了针对类别型特征的统计方法，如下表：

方法名称	说明
value_counts	实现频数统计
describe	描述性统计，如列的非空元素的数目，类别的数目，数目最多的类别，数目最多类别的数目
astype	将非数值型（类别型）数据类型转换为category类别

描述分析DataFrame数据

2. 类别型特征的描述性统计

- 描述类别型特征的分布状况，可以使用频数统计表。pandas库中实现频数统计的方法为value_counts

```
print('订单详情表dishes_name频数统计结果前10为：\n', detail['dishes_name'].value_counts()[0:10])
```

```
订单详情表dishes_name频数统计结果前10为:
 白饭/大碗          91
凉拌菠菜           77
谷稻小庄           72
麻辣小龙虾          65
白饭/小碗          60
五色糯米饭(七色)      58
焖猪手            55
芝士烩波士顿龙虾       55
辣炒鱿鱼           53
水煮鱼            47
Name: dishes_name, dtype: int64
```

对菜品销售数据中的菜品名进行频数统计，因为菜品数目过多，取排名前10的记录。

描述分析DataFrame数据

2. 类别型特征的描述性统计

- **describe方法**除了支持传统数值型以外，还支持非数值型（类别型）数据的描述性统计。例如，统计**列的非空元素的数目，类别的数目，数目最多的类别，数目最多类别的数目。**
- 但在此之前，需要将这些非数值型（类别型）的数据转换为category类型
- **使用astype方法**将这些非数值型（类别型）数据类型转换为category类别。
- 例如，订单表中的菜品不是数值型数据，那按照之前介绍的describe函数，就不能进行“非空值数目、均值、四分位数、标准差。”的统计，但是如果将菜品数据类型转换成category类型，就可以使用describe进行“非空元素的数目，类别的数目，数目最多的类别，数目最多类别的数目”的统计。

描述分析DataFrame数据

2. 类别型特征的描述性统计

（1）将object数据强制转换为category类型

```
detail['dishes_name'] = detail['dishes_name'].astype('category')
print('订单信息表dishes_name列转变数据类型后为：',detail['dishes_name'].dtypes)
```

```
订单信息表dishes_name列转变数据类型后为： category
```


雨课堂 Rain Classroom

描述分析DataFrame数据

2. 类别型特征的描述性统计

（2）使用describe对category类型进行特征的描述性统计

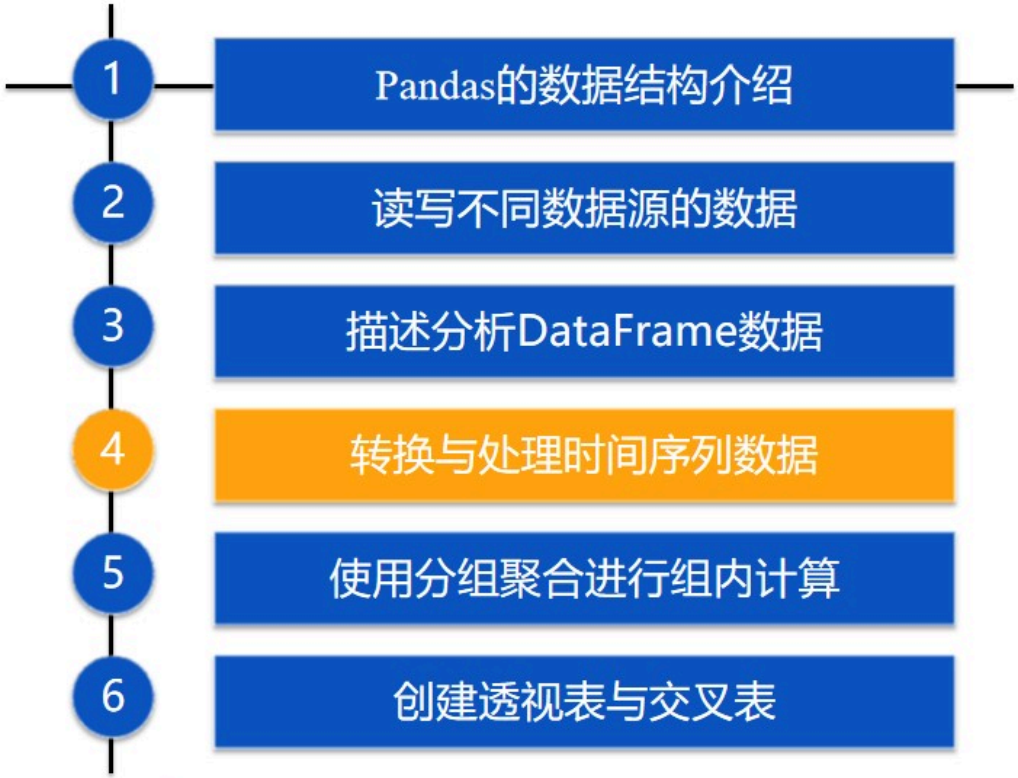
```
print('订单信息表dishes_name的描述统计结果为：\n ',detail['dishes_name'].describe())
```

```
订单信息表dishes_name的描述统计结果为：
 count       2769
unique       145
top       白饭/大碗
freq          91
Name: dishes_name, dtype: object
```

Count：非空元素的数量；
unique：类别的数量；
top：数量最多的类别；
freq：数量最多类别的数量

雨课堂 Rain Classroom

目录


1
Pandas的数据结构介绍
2
读写不同数据源的数据
3
描述分析DataFrame数据
4
转换与处理时间序列数据
5
使用分组聚合进行组内计算
6
创建透视表与交叉表

71

转换与处理时间序列数据

概述

- 数据分析的分析对象不仅局限于数值型和类别型两种，常用的数据类型还包括了时间类型。
- 通过时间类型数据能够获取到对应的年月日和星期等信息。
- 但时间类型数据在读人Python后常常以字符串形式出现，无法实现大部分与时间相关的分析。
- Pandas 库继承了NumPy库的datetime64以及timedelta64模块，能够快速地实现时间字符串的转换、信息提取和时间运算。

雨课堂 Rain Classroom

转换与处理时间序列数据

概述

- 本小节主要完成以下任务：
  - 将订单信息表数据中的时间转换为标准的时间格式；
  - 提取订单信息表数据中的年月日和星期信息；
  - 查看餐饮的数据时间分布

雨课堂 Rain Classroom

转换字符串时间为标准时间

pandas时间相关的类

- 在多数情况下，对时间类型数据进行分析的前提就是**将原本为字符串的时间转换为标准时间类型**。pandas继承了NumPy库和datetime库的时间相关模块，提供了6种时间相关的类。

类名称	说明
Timestamp	特定时刻，最基础的时间类。表示某个时间点。在绝大多数场景中的时间数据都是Timestamp形式的时间。
Period	固定时期，表示单个时间跨度，或者某个时间段，例如某一天，某一小时等。
Timedelta	时间间隔，可以表示两个datetime对象之间的时间差
DatetimeIndex	一组Timestamp构成的Index，可以用来作为Series或者DataFrame的索引。
PeriodtimeIndex	一组Period构成的Index，可以用来作为Series或者DataFrame的索引。
TimedeltaIndex	一组Timedelta构成的Index，可以用来作为Series或者DataFrame的索引。

转换字符串时间为标准时间

Timestamp类型

- Timestamp是时间类中最基础的，也是最常用的。
- 在多数情况下，需要将时间相关的字符串转换成为Timestamp对象。
  - **pandas提供了to_datetime函数**，可以处理成组的日期字符串，解析多种不同的日期表示，非常快

```
In [1]:  1 import pandas as pd

In [2]:  1 datestrs = ['7/6/2011','8/6/2011']

In [3]:  1 pd.to_datetime(datestrs)

Out[3]: DatetimeIndex(['2011-07-06', '2011-08-06'], dtype='datetime64[ns]', freq=None)
```

以ns形式存储时间戳

雨课堂 Rain Classroom

转换字符串时间为标准时间

DatetimeIndex类型

- 跟其他Series一样，不同索引的时间序列之间的算术运算会自动按照日期对齐。

```
In [26]: 1 ts

Out[26]: 2021-10-23    0.256284
         2021-10-25    0.134922
         2021-10-28    0.256270
         2021-10-31   -1.527658
         2021-11-01    1.204220
         2021-11-03    1.231470
         dtype: float64
```

ts每隔一个取一个数据

```
In [28]: 1 ts + ts[::2]

Out[28]: 2021-10-23    0.512568
         2021-10-25         NaN
         2021-10-28    0.512539
         2021-10-31         NaN
         2021-11-01    2.408440
         2021-11-03         NaN
         dtype: float64
```


雨课堂 Rain Classroom

转换字符串时间为标准时间

DatetimeIndex类型

- 由于以DatetimeIndex作为的索引的Series跟其他pandas的Series一样，所以在索引数据以及数据选取等行为方面是一样的。

```
In [29]:  1 stamp = ts.index[3]
          2 ts[stamp]

Out[29]: -1.5276580966093711
```

```
ts
2021-10-23    0.256284
2021-10-25    0.134922
2021-10-28    0.256270
2021-10-31   -1.527658
2021-11-01    1.204220
2021-11-03    1.231470
dtype: float64
```

- 亦可以更为方便的传入一个可以被解释成日期的字符串

```
In [30]:  1 ts['2021-10-28']

Out[30]: 0.2562696072490399
```

转换字符串时间为标准时间

DatetimeIndex类型

- 对于较长的时间序列，只需要传入“年”或者“年月”，就可以轻松的选取数据的切片。
- 首先创建一个较长时间索引的Series，作为测试数据

```
In [33]:  1  longts = pd.Series(np.random.randn(1000),index=pd.date_range('2010-01-01',periods=1000))

In [34]:  1  longts

Out[34]: 2010-01-01    1.582957
         2010-01-02    0.645131
         2010-01-03    1.579827
         2010-01-04    1.675364
         2010-01-05    2.836871
                         ...
         2012-09-22    0.369294
         2012-09-23    1.722097
         2012-09-24   -0.463853
         2012-09-25   -0.044857
         2012-09-26   -0.398595
         Freq: D, Length: 1000, dtype: float64
```

pd.date_range()可以创建DatetimeIndex序列

如下页PPT

转换字符串时间为标准时间

时间序列的基础频率

别名	偏移量类型	说明
D	Day	每日历日
B	BusinessDay	每工作日
H	Hour	每小时
T/min	Minute	每分
S	Second	每秒
M	MonthEnd	每月最后一个日历日
BM	BusinessMonthEnd	每月最后一个工作日
MS	MonthBegin	每月第一个日历日
BMS	BusinessMonthBegin	每月第一个工作日
W-MON、W-TUE.....	Week	从指定的星期几(MON、TUE、WED、THU、FRI、SAT、SUN)开始算起，每周
WOM-1MON.....	WeekOfMonth	每月第几个星期几，例如：WOM-3FRI 每月第三个星期五
Q-JAN、Q-FEB	QuarterEnd	对于指定月份(JAN、FEB、MAR、APR、MAY、JUN、JUL、AUG、SEP、OCT、NOV、DEC)结束的年度，每季度最后一月的最后一个日历日
BQ-JAN、BQ-FEB	BusinessQuarterEnd	对于指定月份结束的年度，每季度最后一月的最后一个工作日

转换字符串时间为标准时间

DatetimeIndex类型

- 对于较长时间索引的Series，只需传入“年”或者“年月”，就可以轻松的选取数据的切片。
- 然后传入“年”进行索引

```
In [35]:  1 longts['2012']

Out[35]: 2012-01-01    1.406979
         2012-01-02    2.220545
         2012-01-03   -0.027644
         2012-01-04    0.435163
         2012-01-05    0.660164
                          ...
         2012-09-22    0.369294
         2012-09-23    1.722097
         2012-09-24   -0.463853
         2012-09-25   -0.044857
         2012-09-26   -0.398595
         Freq: D, Length: 270, dtype: float64
```

```
In [34]:  1 longts

Out[34]: 2010-01-01    1.582957
         2010-01-02    0.645131
         2010-01-03    1.579827
         2010-01-04    1.675364
         2010-01-05    2.836871
                          ...
         2012-09-22    0.369294
         2012-09-23    1.722097
         2012-09-24   -0.463853
         2012-09-25   -0.044857
         2012-09-26   -0.398595
         Freq: D, Length: 1000, dtype: float64
```


雨课堂 Rain Classroom

转换字符串时间为标准时间

DatetimeIndex类型

- 通过日期进行切片的方式，只对规则(排序的)Series有效：

```
In [37]:  1 ts['2021-10-31':]

Out[37]: 2021-10-31   -1.527658
         2021-11-01    1.204220
         2021-11-03    1.231470
         dtype: float64
```

```
ts
2021-10-23    0.256284
2021-10-25    0.134922
2021-10-28    0.256270
2021-10-31   -1.527658
2021-11-01    1.204220
2021-11-03    1.231470
dtype: float64
```

- 由于大部分时间序列数据是按照时间先后排序的，所以可以用不存在于该时间序列中的时间戳进行切片

```
In [38]:  1 ts['2021-10-30':'2021-11-10']

Out[38]: 2021-10-31   -1.527658
         2021-11-01    1.204220
         2021-11-03    1.231470
         dtype: float64
```

转换字符串时间为标准时间

DatetimeIndex类型

- 以上操作对DateFrame也有效，如下先创建一个带有时间索引的DateFrame：

In [56]:
```
dates = pd.date_range('2021-01-01',periods=100,freq='B')
longdf = pd.DataFrame(np.random.rand(100,4),index=dates,columns=['Math','Arts','English','Phy'])
longdf
```

Out[56]:

	Math	Arts	English	Phy
2021-01-01	0.106175	0.064933	0.326112	0.666391
2021-01-04	0.067613	0.075973	0.262707	0.016160
2021-01-05	0.652792	0.874872	0.257203	0.657692
2021-01-06	0.360474	0.035932	0.952531	0.400780
2021-01-07	0.121701	0.563675	0.221895	0.440471
...	...	...	...	...
2021-05-14	0.975224	0.737071	0.028769	0.497407
2021-05-17	0.395872	0.127696	0.444244	0.450042
2021-05-18	0.181804	0.628670	0.501533	0.455751
2021-05-19	0.568147	0.915628	0.592777	0.656985
2021-05-20	0.581496	0.623233	0.059888	0.387992

100 rows × 4 columns

雨课堂 Rain Classroom

转换字符串时间为标准时间

DatetimeIndex类型

- 对DateFrame进行索引访问：

In [56]:
```
1 dates = pd.date_range('2021-01-01',periods=100,freq='
2 longdf = pd.DataFrame(np.random.rand(100,4),index=da
3 longdf
```

Out[56]:

	Math	Arts	English	Phy
2021-01-01	0.106175	0.064933	0.326112	0.666391
2021-01-04	0.067613	0.075973	0.262707	0.016160
2021-01-05	0.652792	0.874872	0.257203	0.657692
2021-01-06	0.360474	0.035932	0.952531	0.400780
2021-01-07	0.121701	0.563675	0.221895	0.440471
...	...	...	...	...
2021-05-14	0.975224	0.737071	0.028769	0.497407
2021-05-17	0.395872	0.127696	0.444244	0.450042
2021-05-18	0.181804	0.628670	0.501533	0.455751
2021-05-19	0.568147	0.915628	0.592777	0.656985
2021-05-20	0.581496	0.623233	0.059888	0.387992

100 rows × 4 columns

In [58]:
```
1 longdf.loc['2021-05']
```

Out[58]:

	Math	Arts	English	Phy
2021-05-03	0.061756	0.988025	0.937683	0.846496
2021-05-04	0.427330	0.806420	0.987098	0.930006
2021-05-05	0.192034	0.084403	0.071031	0.215621
2021-05-06	0.778861	0.079629	0.819516	0.394889
2021-05-07	0.991424	0.847347	0.083753	0.101135
2021-05-10	0.129042	0.697952	0.947377	0.046809
2021-05-11	0.408235	0.438456	0.463798	0.712270
2021-05-12	0.760541	0.905939	0.208102	0.719772
2021-05-13	0.076438	0.560453	0.387975	0.396575
2021-05-14	0.975224	0.737071	0.028769	0.497407
2021-05-17	0.395872	0.127696	0.444244	0.450042
2021-05-18	0.181804	0.628670	0.501533	0.455751
2021-05-19	0.568147	0.915628	0.592777	0.656985
2021-05-20	0.581496	0.623233	0.059888	0.387992

转换字符串时间为标准时间

DatetimeIndex类型

- **带有重复索引**的时间序列：在某些情况下，可能会存在多个数据落在同一个时间点上，如下

In [60]:
```
1 dates = pd.DatetimeIndex(['2021-11-01','2021-11-03','2021-11-03','2021-11-03','2021-11-04'])
2 dup_ts = pd.Series(np.arange(5),index=dates)
```

In [61]:
```
1 dup_ts
```

Out[61]:
```
2021-11-01    0
2021-11-03    1
2021-11-03    2
2021-11-03    3
2021-11-04    4
dtype: int32
```

转换字符串时间为标准时间

DatetimeIndex类型

- 对这个时间序列进行索引，要么产生标量，要么产生切片，具体要看所选的时间索引是否重复

```
In [62]:  1 dup_ts['2021-11-04']

Out[62]: 4

In [63]:  1 dup_ts['2021-11-03']

Out[63]: 2021-11-03    1
         2021-11-03    2
         2021-11-03    3
         dtype: int32
```

```
In [61]:  1 dup_ts

Out[61]: 2021-11-01    0
         2021-11-03    1
         2021-11-03    2
         2021-11-03    3
         2021-11-04    4
         dtype: int32
```


雨课堂 Rain Classroom

转换字符串时间为标准时间

移动时间序列数据（带DatetimeIndex索引数据）

- **移动（shifting）**指的是沿着时间轴将数据前移或后移
- Series和DataFrame都有一个shift方法，用于执行单纯的数据移动，而保持索引不变

```
In [3]:  1 ts = pd.Series(np.random.randn(4), index=pd.date_range('2021-11-6', periods=4, freq="M"))

In [4]:  1 ts

Out[4]: 2021-11-30   -0.214538
        2021-12-31    0.227646
        2022-01-31    0.927567
        2022-02-28    1.367851
        Freq: M, dtype: float64
```

单纯数据移动，会有部分数据丢失，出现NaN

```
In [5]:  1 ts.shift(2)

Out[5]: 2021-11-30         NaN
        2021-12-31         NaN
        2022-01-31   -0.214538
        2022-02-28    0.227646
        Freq: M, dtype: float64
```

```
In [6]:  1 ts.shift(-2)

Out[6]: 2021-11-30    0.927567
        2021-12-31    1.367851
        2022-01-31         NaN
        2022-02-28         NaN
        Freq: M, dtype: float64
```

传入频率，会实现时间戳索引移动，数据不会丢失

```
In [7]:  1 ts.shift(2, freq="M")

Out[7]: 2022-01-31   -0.214538
        2022-02-28    0.227646
        2022-03-31    0.927567
        2022-04-30    1.367851
        Freq: M, dtype: float64
```


雨课堂 Rain Classroom

转换字符串时间为标准时间

period类型

- period（时期类型）表示的是时间区间。例如，数日、数月、数季、数年等。
- 使用pandas的**Period函数创建**（需要一个字符串或整数参数，以及频率参数）

```
In [8]:  1 p = pd.Period(2021,freq = 'A-NOV')

In [9]:  1 p
Out[9]: Period('2021', 'A-NOV')
```

- **Period对象加上或减去一个整数，就可以根据频率进行时期移动**

```
In [10]:  1 p-3
Out[10]: Period('2018', 'A-NOV')
```

```
In [11]:  1 p+2
Out[11]: Period('2023', 'A-NOV')
```

转换字符串时间为标准时间

period类型

- period（时期类型）表示的是时间区间。例如，数日、数月、数季、数年等。
- 使用pandas的**period_range函数可以创建规则的时期范围**

```
In [13]:  1  rng = pd.period_range('2021-09-01','2022-01-01',freq="M")

In [14]:  1  rng

Out[14]: PeriodIndex(['2021-09', '2021-10', '2021-11', '2021-12', '2022-01'], dtype='period[M]', freq='M')
```

PeriodIndex类保存了一组Period，它可以在任何pandas数据结构中被用作索引

转换字符串时间为标准时间

PeriodIndex类型

- PeriodIndex类保存了一组Period，它可以在任何pandas数据结构中被用作索引

```
In [13]: 1 rng = pd.period_range('2021-09-01','2022-01-01',freq="M")

In [14]: 1 rng

Out[14]: PeriodIndex(['2021-09', '2021-10', '2021-11', '2021-12', '2022-01'], dtype='period[M]', freq='M')

In [15]: 1 pd.Series(np.random.randn(5),index=rng)

Out[15]: 2021-09   -0.942477
         2021-10   -1.319479
         2021-11   -0.207693
         2021-12   -0.001651
         2022-01   -1.195902
         Freq: M, dtype: float64
```

转换字符串时间为标准时间

PeriodIndex类型

- 用**to_period方法**，可以将时间戳索引的pandas对象转换为以时期索引的对象

```
In [6]:  1 rng = pd.date_range('2021-01-01',periods=3,freq="M")

In [7]:  1 ts = pd.Series(np.random.randn(3),index=rng)

In [8]:  1 ts

Out[8]: 2021-01-31    1.179501
        2021-02-28   -0.209619
        2021-03-31    0.746818
        Freq: M, dtype: float64
```

```
In [9]:  1 pts = ts.to_period()

In [10]: 1 pts

Out[10]: 2021-01    1.179501
         2021-02   -0.209619
         2021-03    0.746818
         Freq: M, dtype: float64
```

转换字符串时间为标准时间

PeriodIndex类型

- 用**to_timestamp方法**，可以将以时期索引的pandas对象转换为时间戳索引的对象

```
In [20]:  1 pts
```

```
Out[20]: 2021-01    0.470458
         2021-02   -0.868593
         2021-03   -0.826428
         Freq: M, dtype: float64
```

```
In [21]:  1 pts.to_timestamp(how='end')
```

```
Out[21]: 2021-01-31 23:59:59.999999999    0.470458
         2021-02-28 23:59:59.999999999   -0.868593
         2021-03-31 23:59:59.999999999   -0.826428
         dtype: float64
```

加减时间数据

Timedelta类

- Timedelta是时间相关的类中的一个异类，不仅能够使用正数，还能够使用负数表示单位时间，例如1秒，2分钟，3小时等。使用Timedelta类，配合常规的时间相关类能够轻松实现时间的算术运算。目前Timedelta函数中时间周期中没有年和月。所有周期名称，对应单位及其说明如下表所示

周期名称	单位	说明	周期名称	单位	说明
weeks	无	星期	**seconds**	s	秒
days	D	天	**milliseconds**	ms	毫秒
hours	h	小时	**microseconds**	us	微妙
minutes	m	分	**nanoseconds**	ns	纳秒

92

雨课堂 Rain Classroom

加减时间数据

Timedelta类

- 使用Timedelta，可以很轻松地实现在某个时间上加减一段时间。

```
In [9]: time1 = order['lock_time']+pd.Timedelta(days = 1)
   ...: print('lock_time在加上一天前前5行数据为：\n',order['lock_time'][:5])
   ...: print('lock_time在加上一天后前5行数据为：\n',time1[:5])
lock_time在加上一天前前5行数据为：
 0   2016-08-01 11:11:46
1   2016-08-01 11:31:55
2   2016-08-01 12:54:37
3   2016-08-01 13:08:20
4   2016-08-01 13:07:16
Name: lock_time, dtype: datetime64[ns]
lock_time在加上一天后前5行数据为：
 0   2016-08-02 11:11:46
1   2016-08-02 11:31:55
2   2016-08-02 12:54:37
3   2016-08-02 13:08:20
4   2016-08-02 13:07:16
Name: lock_time, dtype: datetime64[ns]
```

93

加减时间数据

Timedelta类

- 除了使用Timedelta实现时间的平移外，还能够直接对两个时间序列进行相减，从而得出一个Timedelta。

```
In [11]: timeDelta = pd.to_datetime('2018-09-26')-order['lock_time']
    ...: print('lock_time距今天的时间间隔数据：\n',timeDelta[:5])
    ...: print('时间间隔数据的数据类型为：',timeDelta.dtypes)
lock_time距今天的时间间隔数据：
 0   785 days 12:48:14
1   785 days 12:28:05
2   785 days 11:05:23
3   785 days 10:51:40
4   785 days 10:52:44
Name: lock_time, dtype: timedelta64[ns]
时间间隔数据的数据类型为： timedelta64[ns]
```


雨课堂 Rain Classroom

提取时间序列数据信息

Timestamp类常用属性

- 在多数涉及时间相关的数据处理、统计分析的过程中，需要提取时间中的年份，月份等数据。使用对应的Timestamp类属性就能够实现这一目的。
- **结合Python列表推导式，可以实现对DataFrame某一列时间信息数据的提取。**

属性名称	说明	属性名称	说明
year	年	week	一年中第几周
month	月	quarter	季节
day	日	weekofyear	一年中第几周
hour	小时	dayofyear	一年中的第几天
minute	分钟	dayofweek	一周第几天
second	秒	weekday	一周第几天
date	日期	weekday_name	星期名称
time	时间	is_leap_year	是否闰年

95

雨课堂 Rain Classroom

提取时间序列数据信息

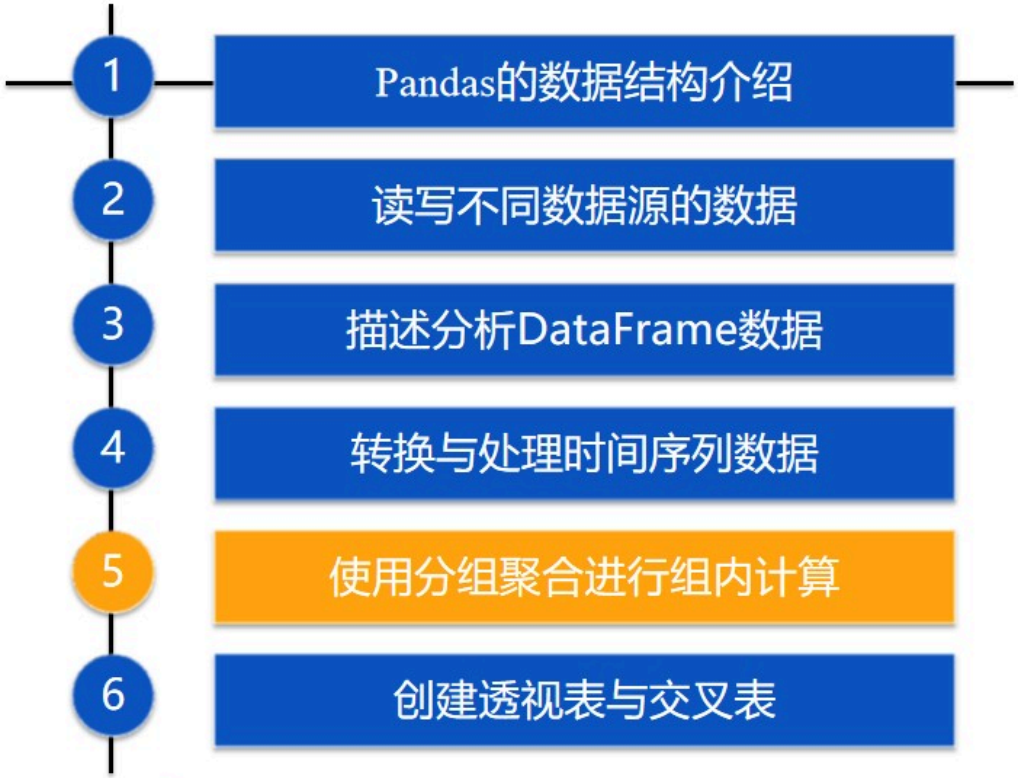
提取datetime数据中的时间序列数据

```
year1 = [i.year for i in order['lock_time']]
print('lock_time中的年份数据前5个为：',year1[:5])
month1 = [i.month for i in order['lock_time']]
print('lock_time中的月份数据前5个为：',month1[:5])
day1 = [i.day for i in order['lock_time']]
print('lock_time中的日期数据前5个为：',day1[:5])
weekday1 = [i.weekday_name for i in order['lock_time']]
print('lock_time中的星期名称数据前5个为：',weekday1[:5])
```

```
lock_time中的年份数据前5个为： [2016, 2016, 2016, 2016, 2016]
lock_time中的月份数据前5个为： [8, 8, 8, 8, 8]
lock_time中的日期数据前5个为： [1, 1, 1, 1, 1]
lock_time中的星期名称数据前5个为： ['Monday', 'Monday', 'Monday', 'Monday', 'Monday']
```

96

目录


1
Pandas的数据结构介绍
2
读写不同数据源的数据
3
描述分析DataFrame数据
4
转换与处理时间序列数据
5
使用分组聚合进行组内计算
6
创建透视表与交叉表

使用分组聚合进行组内计算

任务描述

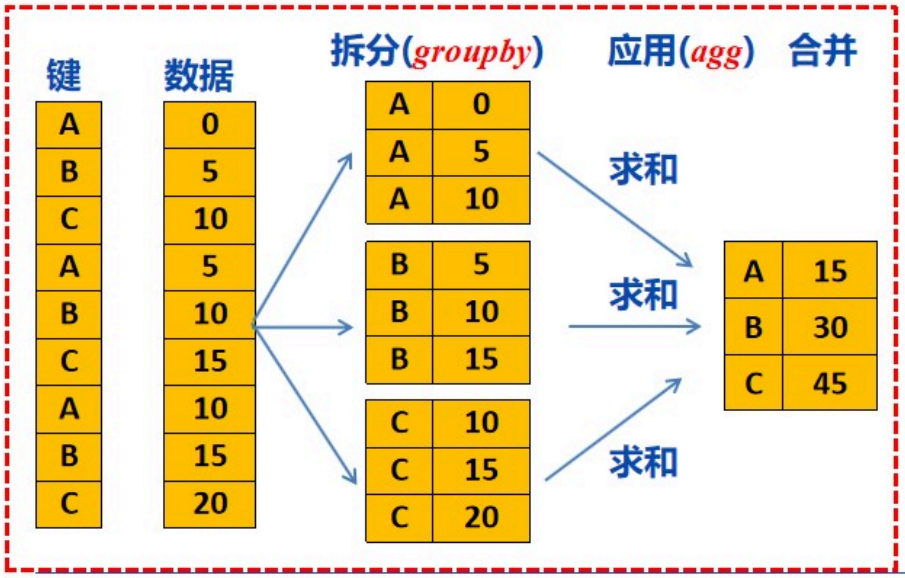
- 依据某个或者某几个字段对数据集进行分组，并对各组应用一个函数（比如聚合或转换），这是数据分析工作的重要环节。
- 通常数据分析工作，就是在数据集准备好之后，**“计算分组统计”或“生成透视表”**。
- 本小节主要介绍“分组统计”：
  - Pandas提供了一个灵活高效的groupby方法，配合agg方法和apply方法，能够实现分组聚合等操作。

使用分组聚合进行组内计算

任务描述

- Pandas提供了一个灵活高效的groupby方法，配合agg方法和apply方法，能够实现分组聚合等操作。

- 分组聚合的原理如图



使用groupby方法拆分数据

任务分析

（1）按照时间对菜品订单数据进行拆分

（2）使用agg方法计算单日菜品销售的总额（agg方法可以对不同列应用不同方法）

（3）使用apply方法统计单日菜品销售数目（apply方法对所有列使用相同的方法）

（4）使用transform方法聚合数据

雨课堂 Rain Classroom

使用groupby方法拆分数据

groupby方法的参数及其说明

- 提供的是分组聚合步骤中的拆分功能，能根据索引或字段对数据进行分组。其使用格式如下：

*DataFrame.groupby(by=None, axis=0, level=None, as_index=True, sort=True, group_keys=True, squeeze=False, **kwargs)*

雨课堂 Rain Classroom

使用groupby方法拆分数据

对菜品订单详情表依据订单编号分组

```
import pandas as pd
import numpy as np
from sqlalchemy import create_engine
engine = create_engine('mysql+pymysql://root:1234@127.0.0.1:3306/testdb?charset=utf8')
detail = pd.read_sql_table('meal_order_detail1',con = engine)
detailGroup = detail[['order_id','counts','amounts']].groupby(by = 'order_id')
print('分组后的订单详情表为：',detailGroup)
```

```
分组后的订单详情表为： <pandas.core.groupby.groupby.DataFrameGroupBy object at 0x0000000009F70E80>
```

从运行结果看出，分组后的结果并不能直接查看，而是被存在内存中，输出的是内存地址。

实际上，分组后的数据对象GroupBy类似于Series或DataFrame，是pandas提供的一种对象。

雨课堂 Rain Classroom

使用groupby方法拆分数据

GroupBy对象常用的描述性统计方法

- 那怎么看结果呢，结果什么样呢？可以通过它常用的描述性方法来观察。这些方法为查看每一组数据的整体情况、分布状态提供了良好的支撑。
- **GroupBy对象**常用的描述性统计方法如下：

方法名称	说明	方法名称	说明
count	计算分组的数目，包括缺失值。	cumcount	对每个分组中的组员进行标记，0至n-1。
head	返回每组的前n个值。	size	返回每组的大小。
max	返回每组最大值。	min	返回每组最小值。
mean	返回每组的均值。	std	返回每组的标准差。
median	返回每组的中位数。	sum	返回每组的和。

使用groupby方法拆分数据

对菜品订单详情表分组后统计每一组的均值、标准差、中位数等

```
print('订单详情表分组后前5组每组的均值为：\n', detailGroup.mean().head())
print('订单详情表分组后前5组每组的标准差为：\n', detailGroup.std().head())
print('订单详情表分组后前5组每组的大小为：\n', detailGroup.size().head())
```

```
订单详情表分组后前5组每组的均值为：
           counts  amounts
order_id
1002       1.0000   32.000
1003       1.2500   30.125
1004       1.0625   43.875
1008       1.0000   63.000
1011       1.0000   57.700
```

```
订单详情表分组后前5组每组的标准差为：
           counts     amounts
order_id
1002       0.00000  16.000000
1003       0.46291  21.383822
1004       0.25000  31.195886
1008       0.00000  64.880660
1011       0.00000  50.077828
```

```
订单详情表分组后前5组每组的大小为：
 order_id
1002     7
1003     8
1004    16
1008     5
1011    10
dtype: int64
```


雨课堂 Rain Classroom

使用agg方法聚合数据

agg函数参数及其说明

- agg支持对每个分组应用某函数，包括Python内置函数或自定义函数。同时这两个方法也能够直接对DataFrame进行函数应用操作。

*DataFrame.agg(func, axis=0, *args, **kwargs)*

参数名称	说明
func	接收list、dict、function。表示应用于每行 / 每列的函数。无默认。
axis	接收0或1。代表操作的轴向。默认为0。

雨课堂 Rain Classroom

使用agg方法聚合数据

agg方法求统计量（对DataFrame数据）

- 可以使用agg方法一次求出当前数据中所有菜品销量和售价的总和与均值，如
  `detail[['counts','amounts']].agg([np.sum,np.mean])`
- 对于某个字段希望只做求均值操作，而对另一个字段则希望只做求和操作，可以使用字典的方式，将两个字段名分别作为key，然后将NumPy库的求和与求均值的函数分别作为value，如
  `detail.agg({'counts':np.sum,'amounts':np.mean})`
- 在某些时候还希望求出某个字段的多个统计量，某些字段则只需要求一个统计量，此时只需要将字典对应key的value变为列表，列表元素为多个目标的统计量即可，如
  `detail.agg({'counts':np.sum,'amounts':[np.mean,np.sum]})`

使用agg方法聚合数据

agg方法求统计量（对DataFrame数据）

detail[['counts','amounts']].agg([np.sum,np.mean])

订单详情表的菜品销量与售价的和与均值为：

	counts	amounts
sum	3087.000000	125833.000000
mean	1.111231	45.296256

detail.agg({'counts':np.sum,'amounts':np.mean})

订单详情表的菜品销量总和与售价的均值为：

```
counts      3087.000000
amounts       45.296256
dtype: float64
```

detail.agg({'counts':np.sum,'amounts':[np.mean,np.sum]})

菜品订单详情表的菜品销量总和与售价的总和与均值为：

	counts	amounts
mean	NaN	45.296256
sum	3087.0	125833.000000

107

使用agg方法聚合数据

agg方法与自定义的函数（对DataFrame数据）

- 在agg方法可传入读者自定义的函数。

```
##自定义函数求两倍的和
def DoubleSum(data):
    s = data.sum()*2
    return s
print('菜品订单详情表的菜品销量两倍总和为：\n', detail.agg({'counts':DoubleSum},axis = 0))
```

```
菜品订单详情表的菜品销量两倍总和为：
 counts    6174.0
dtype: float64
```

使用agg方法聚合数据

之前讲的都是对DataFrame数据进行描述性统计，其实这些方法也适用于分组的描述性统计。

agg方法与自定义的函数（对GroupBy数据）

- 使用agg方法能够实现对每一个字段每一组使用相同的函数。

```
print('订单详情表分组后前3组每组的均值为：\n', detailGroup.agg(np.mean).head(3))
print('订单详情表分组后前3组每组的标准差为：\n', detailGroup.agg(np.std).head(3))
```

订单详情表分组后前3组每组的均值为：

order_id	counts	amounts
1002	1.0000	32.000
1003	1.2500	30.125
1004	1.0625	43.875

订单详情表分组后前3组每组的标准差为：

order_id	counts	amounts
1002	0.00000	16.000000
1003	0.46291	21.383822
1004	0.25000	31.195886

使用agg方法聚合数据

agg方法与自定义的函数（对GroupBy数据）

- 如果需要对不同的字段应用不同的函数，则可以和DataFrame中使用agg方法相同。

```
print('订单详情分组前3组每组菜品总数和售价均值为：\n',
      detailGroup.agg({'counts':np.sum, 'amounts':np.mean}).head(3))
```

```
In [21]: print('订单详情分组前3组每组菜品总数和售价均值为：\n',
    ...:       detailGroup.agg({'counts':np.sum,'amounts':np.mean}).head(3))
订单详情分组前3组每组菜品总数和售价均值为：
           counts  amounts
order_id
1002          7.0   32.000
1003         10.0   30.125
1004         17.0   43.875
```

- 使用agg方法对分组后的菜品订单表求每组菜品总数和售价均值。

雨课堂 Rain Classroom

使用apply方法聚合数据

- apply方法类似agg方法，能够将函数应用于每一列。
- **不同之处**在于apply方法传入的函数只能够作用于整个DataFrame或者Series，而无法像agg一样能够对不同字段，应用不同函数获取不同结果。

*DataFrame.apply(func, axis=0, broadcast=False, raw=False, reduce=None, args=(), **kwds)*

- 实例：

```
print('订单详情表的菜品销量与售价的均值为：\n',
      detail[['counts','amounts']].apply(np.mean))
```

```
订单详情表的菜品销量与售价的均值为：
 counts      1.111231
amounts     45.296256
dtype: float64
```

使用apply方法聚合数据

- 使用apply方法**对GroupBy对象**进行聚合操作其方法和agg方法也相同
- **print('订单详情表分组后前3组每组的均值为：\n',detailGroup.apply(np.mean).head(3))**

  print('订单详情表分组后前3组每组的标准差为：\n',detailGroup.apply(np.std).head(3))

```
订单详情表分组后前3组每组的均值为：
              order_id  counts  amounts
order_id
1002      1.431572e+26  1.0000   32.000
1003      1.253875e+30  1.2500   30.125
1004      6.275628e+61  1.0625   43.875
订单详情表分组后前3组每组的标准差为：
             counts    amounts
order_id
1002       0.000000  14.813122
1003       0.433013  20.002734
1004       0.242061  30.205287
```

使用transform方法

- transform方法能够**对整个DataFrame**的**所有元素**进行操作（不一定是聚合操作）。
- 且transform方法只有一个参数“func”，表示对DataFrame操作的函数。

```
print('订单详情表的菜品销量与售价的两倍为：\n',
      detail[['counts','amounts']].transform(lambda x:x*2).head(4))
```

```
订单详情表的菜品销量与售价的两倍为：
    counts  amounts
0      2.0     98.0
1      2.0     96.0
2      2.0     60.0
3      2.0     50.0
```

使用transform方法

- 同时transform方法还能够**对DataFrame分组后的对象GroupBy**进行操作，可以实现组内离差标准化等操作。

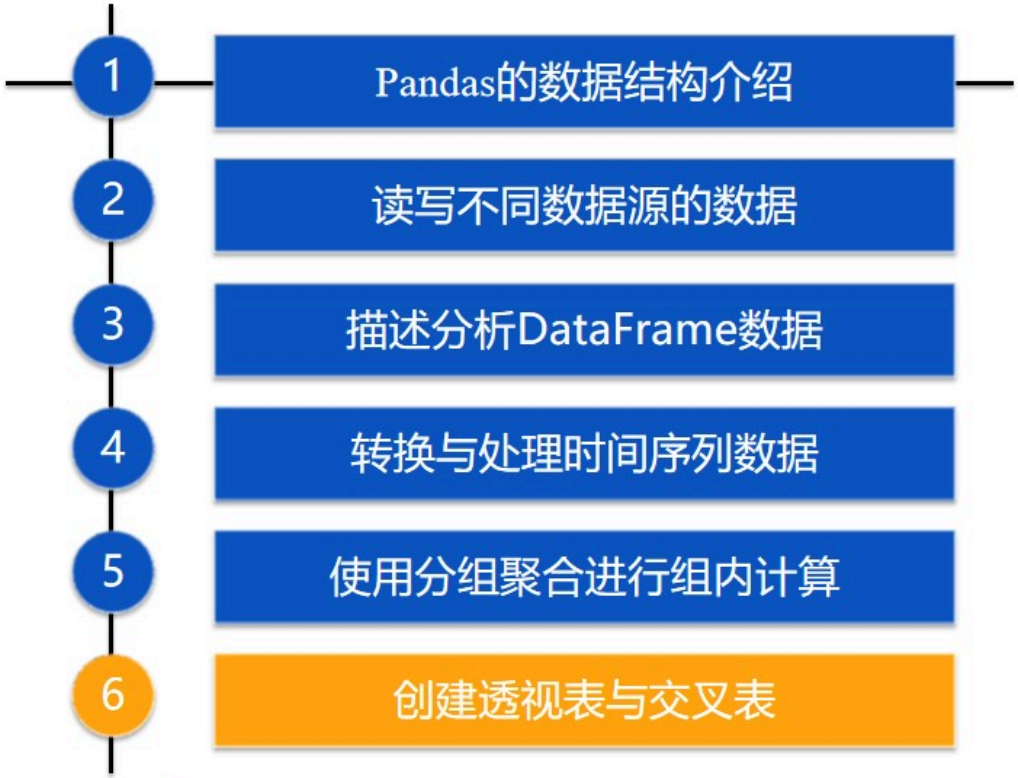
```
print('订单详情表分组后实现组内离差标准化后的前五行为：\n',
      detailGroup.transform(lambda x:(x.mean()-x.min())/(x.max()-x.min())).head())
```

```
__main__:2: RuntimeWarning: invalid value encountered in double_scalars
订单详情表分组后实现组内离差标准化后的前五行为：
    counts   amounts
0      NaN  0.555556
1      NaN  0.555556
2      NaN  0.555556
3      NaN  0.555556
4      NaN  0.555556
```

在计算离差标准化的时候结果中有NaN，这是由于根据离差标准化公式，最大值和最小值相同的情况下分母是0。而分母为0的数在Python中表示为NaN。

雨课堂 Rain Classroom

目录


1
Pandas的数据结构介绍
2
读写不同数据源的数据
3
描述分析DataFrame数据
4
转换与处理时间序列数据
5
使用分组聚合进行组内计算
6
创建透视表与交叉表

创建透视表与交叉表

任务描述

- 数据透视表是数据分析中常见的工具之一，根据一个或多个键值对数据进行聚合，根据行或者列的分组将数据划分到各个区域。
- 在pandas中，除了可以使用groupby对数据分组聚合实现透视功能外，还提供了更为简单的方法。
- 本小节以餐饮企业的菜品订单数据为例制作透视表和交叉表，分析不同菜品的销售和金额之间的关系。

任务分析

（1）使用pivot_table函数制作菜品日销售透视表

（2）使用crosstab函数制作菜品销售的交叉表

使用pivot_table函数创建透视表

使用订单号作为透视表索引制作透视表

```
import pandas as pd
import numpy as np
from sqlalchemy import create_engine
engine =
    create_engine('mysql+pymysql://root:1234@127.0.0.1:3306/testdb?charset=utf8')
detail = pd.read_sql_table('meal_order_detail1',con = engine)

detailPivot = pd.pivot_table(detail[['order_id','counts','amounts']], index = 'order_id')
print('以order_id作为分组键创建的订单透视表为：\n', detailPivot.head())
```

117

使用pivot_table函数创建透视表

使用订单号作为透视表索引制作透视表

```
import pandas as pd
import numpy as np
from sqlalchemy import create_engine
engine =
    create_engine('mysql+pymysql://root:1234@127.0.0.1:3306/testdb?charset=utf8')
detail = pd.read_sql_table('meal_order_de
```

```
以order_id作为分组键创建的订单透视表为：
          amounts  counts
order_id
1002       32.000  1.0000
1003       30.125  1.2500
1004       43.875  1.0625
1008       63.000  1.0000
1011       57.700  1.0000
```

- 从运行结果看出，当不特殊指定聚合函数aggfunc时，会默认使用numpy.mean进行聚合运算
- numpy.mean会自动过滤掉非数值类型数据。

```
detailPivot = pd.pivot_table(detail[['order_id','counts','amounts']], index = 'order_id')
print('以order_id作为分组键创建的订单透视表为：\n', detailPivot.head())
```

使用pivot_table函数创建透视表

pivot_table函数主要的参数调节

- 在不特殊指定聚合函数aggfunc时，会默认使用numpy.mean进行聚合运算，numpy.mean会自动过滤掉非数值类型数据。

```
In [42]: detailPivot = pd.pivot_table(detail[['order_id','counts','amounts','dishes_name']],
    ...:                              index = 'order_id')
    ...: print('以order_id作为分组键创建的订单透视表为：\n',detailPivot.head())
以order_id作为分组键创建的订单透视表为：
            amounts  counts
order_id
1002        32.000  1.0000
1003        30.125  1.2500
1004        43.875  1.0625
1008        63.000  1.0000
1011        57.700  1.0000
```

使用pivot_table函数创建透视表

pivot_table函数主要的参数调节

- 可以通过指定aggfunc参数修改聚合函数：

```
detailPivot1 = pd.pivot_table(detail[['order_id','counts','amounts']],
                              index = 'order_id', aggfunc = np.sum)
print('以order_id作为分组键创建的订单销量与售价总和透视表为：\n', detailPivot1.head())
```

```
以order_id作为分组键创建的订单销量与售价总和透视表为：
          amounts  counts
order_id
1002        224.0     7.0
1003        241.0    10.0
1004        702.0    17.0
1008        315.0     5.0
1011        577.0    10.0
```

使用pivot_table函数创建透视表

pivot_table函数主要的参数调节

- 和groupby方法分组的时候相同，pivot_table函数在创建透视表的时候分组键index可以有多个。

```
detailPivot2 = pd.pivot_table(detail[['order_id','dishes_name','counts','amounts']],
                              index = ['order_id','dishes_name'], aggfunc = np.sum)
print('以order_id和dishes_name作为分组键创建的订单销量与售价总和透视表为：\n',
      detailPivot2.head())
```

```
以order_id和dishes_name作为分组键创建的订单销量与售价总和透视表为：
                        amounts  counts
order_id dishes_name
1002     凉拌菠菜              27.0     1.0
         南瓜枸杞小饼干          19.0     1.0
         焖猪手               58.0     1.0
         独家薄荷鲜虾牛肉卷       45.0     1.0
         白胡椒胡萝卜羊肉汤       35.0     1.0
```

使用pivot_table函数创建透视表

pivot_table函数主要的参数调节

- 当全部数据列数很多时，若只想要显示某列，可以通过指定values参数来实现。

```
detailPivot4 = pd.pivot_table(detail[['order_id','dishes_name','counts','amounts']],
                              index = 'order_id', values = 'counts', aggfunc = np.sum)
print('以order_id作为行分组键counts作为值创建的透视表前5行为：\n',detailPivot4.head())
```

```
以order_id作为行分组键counts作为值创建的透视表前5行为：
            counts
order_id
1002          7.0
1003         10.0
1004         17.0
1008          5.0
1011         10.0
```

使用crosstab函数创建交叉表

crosstab函数

- 交叉表是一种特殊的透视表，主要用于**计算分组频率**
- 利用pandas提供的**crosstab函数**可以制作交叉表
- crosstab函数的常用参数和使用格式如下：

```
pandas.crosstab(index, columns, values=None, rownames=None, colnames=None, aggfunc=None,
                margins=False, dropna=True, normalize=False)
```

123

雨课堂 Rain Classroom

使用crosstab函数创建交叉表

crosstab函数

- 由于交叉表是透视表的一种，其参数基本保持一致，不同之处在于crosstab函数进行的操作是以index为分组，columns和values交叉元素进行aggfunc的聚合操作。

```
detailCross = pd.crosstab(index=detail['order_id'], columns=detail['dishes_name'],
                          values = detail['counts'], aggfunc = np.sum)
print('以order_id和dishes_name为分组键counts为值的透视表前5行5列为：\n',
      detailCross.iloc[:5,:5])
```

```
以order_id和dishes_name为分组键counts为值的透视表前5行5列为：
 dishes_name    42度海之蓝    北冰洋汽水    38度剑南春    50度古井贡酒    52度泸州老窖
order_id
1002              NaN       NaN       NaN       NaN       NaN
1003              NaN       NaN       NaN       NaN       NaN
1004              NaN       NaN       NaN       NaN       NaN
1008              NaN       NaN       NaN       NaN       NaN
1011              1.0       NaN       NaN       NaN       NaN
```

透视表交叉表使用示例（自我练习）

1. 创建单日菜品成交总额与总数均价透视表

- 单日菜品的成交总额以及菜品成交总数对于营业状况分析、利润分析具有重要作用，使用透视表能够轻松实现这一操作。

```
import pandas as pd
import numpy as np
from sqlalchemy import create_engine
engine = create_engine('mysql+pymysql://root:1234@127.0.0.1:3306/testdb?charset=utf8')
detail = pd.read_sql_table('meal_order_detail1',con = engine)
detail['place_order_time'] = pd.to_datetime(detail['place_order_time'])
detail['date'] = [i.date() for i in detail['place_order_time']]
PivotDetail = pd.pivot_table(detail[['date','dishes_name','counts','amounts']],
                             index ='date', aggfunc = np.sum, margins = True)
print('订单详情表单日菜品成交总额与总数透视表前5行5列为：\n', PivotDetail.head())
```

125

任务实现（自我练习）

1.创建单日菜品成交总额与总数均价透视表

- 单日菜品的成交总额以及菜品成交总数对于营业状况分析、利润分析具有重要作用，使用透视表能够轻松实现这一操作。

订单详情表单日菜品成交总额与总数透视表前5行5列为：

date	amounts	counts
2016-08-01	9366.0	233.0
2016-08-02	6125.0	151.0
2016-08-03	6890.0	192.0
2016-08-04	7549.0	169.0
2016-08-05	8671.0	224.0

雨课堂 Rain Classroom

任务实现（自我练习）

2.创建单个菜品单日成交总额交叉表

- 单个菜品单日的成交总额对后续菜品价格调整，以及菜品种类调整具有十分重要的作用，使用交叉表能够轻松实现。

```
CrossDetail = pd.crosstab( index=detail['date'], columns=detail['dishes_name'],
                           values = detail['amounts'], aggfunc = np.sum, margins = True)
print('订单详情表单日单个菜品成交总额交叉表后5行5列为：\n', CrossDetail.iloc[-5:,-5:])
```

```
订单详情表单日单个菜品成交总额交叉表后5行5列为：
 dishes_name  黄尾袋鼠西拉子红葡萄酒  黄油曲奇饼干  黄花菜炒木耳  黑米恋上葡萄      All
date
2016-08-07          230.0    32.0   105.0    99.0   31306.0
2016-08-08           46.0     NaN     NaN    33.0    6532.0
2016-08-09          138.0     NaN    35.0    99.0    7155.0
2016-08-10           46.0     NaN    70.0    33.0   10231.0
All                 736.0    80.0   525.0   561.0  125833.0
```


雨课堂 Rain Classroom

小结

本章以餐饮数据为例

- 介绍了数据库数据，csv数据，Excel数据三种常用的数据读取与写入方式。
- 阐述了DataFrame的常用属性，方法与描述性统计相关内容。
- 介绍了时间数据的转换，信息提取与算术运算。
- 剖析了分组聚合方法groupby的原理，用法和三种聚合方法。
- 展现了透视表与交叉表的制作方法。

通过本章的学习，读者能够对pandas库有一个整体了解并能够利用pandas库进行基础的统计。

雨课堂 Rain Classroom

使用Pandas进行数据预处理

2026/10/15

目录

合并数据

任务描述

- 菜品数据表格数据非常多，如菜品订单详情表、菜品信息表、菜品分类表和用户信息表等，这些数据分别存储了菜品数据分析过程中所需要的各种信息。
- 通过堆叠合并和主键合并等多种合并方式，可以将关联的数据信息合并在一张表中。

任务分析

（1）横向或纵向堆叠合并数据

（2）主键合并数据

（3）重叠合并数据

- 堆叠就是简单地把两个表拼在一起，也被称作轴向连接、绑定或连接。
- 依照连接轴的方向，数据堆叠可分为横向堆叠和纵向堆叠。

堆叠合并数据

横向或纵向堆叠合并数据——concat函数

- concat函数基本语法如下：

```
pandas.concat(objs, axis=0, join='outer', join_axes=None, ignore_index=False,
              keys=None, levels=None, names=None, verify_integrity=False, copy=True)
```


雨课堂 Rain Classroom

堆叠合并数据

1. 横向堆叠 —— concat函数（axis=1）

- concat将两张或多张表的**所有列名称合并**(特征堆叠)，行索引根据join参数进行'内/外'连接。

  pandas.concat([df1,df2], axis=1)

- 缺省外连接： join = 'outer'，行索引取并集，缺失数据用空值填补。

dataframe1

	A	B	C	D
1	A1	B1	C1	D1
2	A2	B2	C2	D2
3	A3	B3	C3	D3
4	A4	B4	C4	D4

dataframe2

	B	D	F
2	B2	D2	F2
4	B4	D4	F4
6	B6	D6	F6
8	B8	D8	F8

dataframe3

	A	B	C	D	B	D	F
1	A1	B1	C1	D1	NaN	NaN	NaN
2	A2	B2	C2	D2	B2	D2	F2
3	A3	B3	C3	D3	NaN	NaN	NaN
4	A4	B4	C4	D4	B4	D4	F4
6	NaN	NaN	NaN	NaN	B6	D6	F6
8	NaN	NaN	NaN	NaN	B8	D8	F8

堆叠合并数据

1. 横向堆叠 —— concat函数（axis=1）

- concat将两张或多张表的所有列名称合并(特征堆叠)，行索引根据join参数进行'内/外'连接。

  pandas.concat([df1,df2], axis=1, join = 'inner')

- 内连接时，行索引取交集，不会出现缺失值。

dataframe1

	A	B	C	D
1	A1	B1	C1	D1
2	A2	B2	C2	D2
3	A3	B3	C3	D3
4	A4	B4	C4	D4

dataframe2

	B	D	F
2	B2	D2	F2
4	B4	D4	F4
6	B6	D6	F6
8	B8	D8	F8

dataframe3

	A	B	C	D	B	D	F
2	A2	B2	C2	D2	B2	D2	F2
4	A4	B4	C4	D4	B4	D4	F4

雨课堂 Rain Classroom

堆叠合并数据

1. 横向堆叠 —— concat函数（axis=1）

- concat将两张或多张表的所有列名称合并(特征堆叠)，行索引根据join参数进行'内/外'连接。

  pandas.concat([df2,df2], axis=1, join = 'outer')

  pandas.concat([df2,df2], axis=1, join = 'inner')

- 当两张表行索引完全一样时，不论join参数取值是inner或者outer，结果都是将两个表完全按照X轴拼接起来。

dataframe1

	B	D	F
2	B1	D1	F1
4	B3	D3	F3
6	B5	D5	F5
8	B7	D7	F7

dataframe2

	B	D	F
2	B2	D2	F2
4	B4	D4	F4
6	B6	D6	F6
8	B8	D8	F8

dataframe3

	B	D	F	B	D	F
2	B1	D1	F1	B2	D2	F2
4	B3	D3	F3	B4	D4	F4
6	B5	D5	F5	B6	D6	F6
8	B7	D7	F7	B8	D8	F8

堆叠合并数据（自我练习）

1. 横向表堆叠

```
import numpy as np
import pandas as pd
from sqlalchemy import create_engine
conn = create_engine('mysql+pymysql://root:1234@127.0.0.1:3306/testdb?charset=utf8')
detail1 = pd.read_sql('meal_order_detail1',conn)
df1 = detail1.iloc[:, :10]                    #取出detail1的前10列数据
df2 = detail1.iloc[:, 10:]                    #取出detail1的后9列数据
print('合并df1的大小为%s，df2的大小为%s。'%(df1.shape,df2.shape))
print('外连接合并后的数据框大小为：',pd.concat([df1,df2], axis=1,join='inner').shape)
print('内连接合并后的数据框大小为：',pd.concat([df1,df2], axis=1,join='outer').shape)
```

堆叠合并数据（自我练习）

1. 横向表堆叠

```
import numpy as np
import pandas as pd
from sqlalchemy import create_engine
conn = create_engine('mysql+pymysql://root:1234@127.0.0.1:3306/testdb?charset=utf8')
detail1 = pd.read_sql('meal_order_detail1',conn)
df1 = detail1.iloc[:, :10]                    #取出detail1的前10列数据
df2 = detail1.iloc[:, 10:]                    #取出detail1的后9列数据
print('合并df1的大小为%s，df2的大小为%s。'%(df1.shape,df2.shape))
print('外连接合并后的数据框大小为：',pd.concat([df1,df2], axis=1,join='inner').shape)
print('内连接合并后的数据框大小为：',pd.concat([df1,df2], axis=1,join='outer').shape)
```

```
合并df1的大小为(2779, 10)，df2的大小为(2779, 9)。
外连接合并后的数据框大小为： (2779, 19)
内连接合并后的数据框大小为： (2779, 19)
```

137

堆叠合并数据

2. 纵向堆叠 —— concat函数（axis=0）

- concat函数在默认情况下做纵向合并(记录堆叠)，将两张或多张表的所有行索引名称堆叠，列标签根据join参数进行'内/外'连接。

  pandas.concat([df1,df2], axis=0)

- 缺省外连接： join = 'outer'，

  列标签取并集，缺失数据用空值填补。

dataframe1

	A	B	C	D
1	A1	B1	C1	D1
2	A2	B2	C2	D2
3	A3	B3	C3	D3
4	A4	B4	C4	D4

dataframe2

	B	D	F
2	B2	D2	F2
4	B4	D4	F4
6	B6	D6	F6
8	B8	D8	F8

dataframe3

	A	B	C	D	F
1	A1	B1	C1	D1	NaN
2	A2	B2	C2	D2	NaN
3	A3	B3	C3	D3	NaN
4	A4	B4	C4	D4	NaN
2	NaN	B2	NaN	D2	F2
4	NaN	B4	NaN	D4	F4
6	NaN	B6	NaN	D6	F6
8	NaN	B8	NaN	D8	F8

堆叠合并数据

2. 纵向堆叠 —— concat函数（axis=0）

- concat函数在默认情况下做纵向合并(记录堆叠)，将两张或多张表的所有行索引名称堆叠，列标签根据join参数进行'内/外'连接。

  pandas.concat([df1,df2], axis=0, join='inner')

- 内连接时，列标签取交集,不会出现缺失值。

dataframe1

	A	B	C	D
1	A1	B1	C1	D1
2	A2	B2	C2	D2
3	A3	B3	C3	D3
4	A4	B4	C4	D4

dataframe2

	B	D	F
2	B2	D2	F2
4	B4	D4	F4
6	B6	D6	F6
8	B8	D8	F8

dataframe3

	B	D
1	B1	D1
2	B2	D2
3	B3	D3
4	B4	D4
2	B2	D2
4	B4	D4
6	B6	D6
8	B8	D8

139

堆叠合并数据

2. 纵向堆叠 —— concat函数（axis=0）

- concat函数在默认情况下做纵向合并(记录堆叠)，将两张或多张表的所有行索引名称堆叠，列标签根据join参数进行'内/外'连接。

  pandas.concat([df1,df2], axis=0, join='outer')

  pandas.concat([df1,df2], axis=0, join='inner')

- 在两张表的列名完全相同的情况下，不论join参数取值是inner或者outer，结果都是将两个表完全按照Y轴拼接起来

dataframe1

	B	D	F
1	B1	D1	F1
2	B2	D2	F2
3	B3	D3	F3
4	B4	D4	F4

dataframe2

	B	D	F
2	B2	D2	F2
4	B4	D4	F4
6	B6	D6	F6
8	B8	D8	F8

dataframe3

	B	D	F
1	B1	D1	F1
2	B2	D2	F2
3	B3	D3	F3
4	B4	D4	F4
2	B2	D2	F2
4	B4	D4	F4
6	B6	D6	F6
8	B8	D8	F8

堆叠合并数据（自我练习）

2. 纵向堆叠 —— concat函数（axis=0）

```
df3 = detail1.iloc[:1500,:]          #取出detail1前1500行数据
df4 = detail1.iloc[1500:,:]          #取出detail1的1500行之后的数据
print('合并df3的大小为%s，df4的大小为%s。'%(df3.shape,df4.shape))
print('内连接纵向合并后的数据框大小为：',pd.concat([df3,df4], axis=0, join='inner').shape)
print('外连接纵向合并后的数据框大小为：',pd.concat([df3,df4], axis=0,join='outer').shape)
```

```
合并df3的大小为(1500, 19)，df4的大小为(1279, 19)。
内连接纵向合并后的数据框大小为： (2779, 19)
外连接纵向合并后的数据框大小为： (2779, 19)
```


雨课堂 Rain Classroom

堆叠合并数据

2. 纵向堆叠 —— append方法

- append方法也可以用于**纵向合并两张表**。但有一个前提条件，那就是**两张表的列名需要完全一致**。append方法的基本语法如下：

pandas.DataFrame.append(self, other, ignore_index=False, verify_integrity=False)

- 常用参数如下所示：

参数名称	说明
other	接收DataFrame或Series。表示要添加的新数据。无默认。
ignore_index	接收boolean。如果输入True，会对新生成的DataFrame使用新的索引（自动产生）而忽略原来数据的索引。默认为False。
verify_integrity	接收boolean。如果输入True，那么当ignore_index为False时，会检查添加的数据索引是否冲突，如果冲突，则会添加失败。默认为False。

堆叠合并数据（自我练习）

2. 纵向堆叠 —— append方法

```
print('堆叠前df3的大小为%s，df4的大小为%s。'%(df3.shape,df4.shape))
print('append纵向堆叠后的数据框大小为：',df3.append(df4).shape)
```

```
堆叠前df3的大小为(1500, 19)，df4的大小为(1279, 19)。
append纵向堆叠后的数据框大小为： (2779, 19)
```


雨课堂 Rain Classroom

主键合并数据

主键合并 —— merge函数

- 主键合并，即通过一个或多个键将两个数据集的行连接起来，类似于SQL中的JOIN。
- merge函数有左连接（left）、右连接（right）、内连接（inner）和外连接（outer）.
- 可以在合并过程中对数据集中的数据进行排序等。

```
pandas.merge(left, right, how='inner', on=None, left_on=None, right_on=None,
             left_index=False, right_index=False, sort=False, suffixes=('_x', '_y'),
             copy=True, indicator=False)
```

- 可根据merge函数中的参数说明，并按照需求修改相关参数，就可以多种方法实现主键合并。

雨课堂 Rain Classroom

主键合并数据

主键合并 —— merge函数

pd.merge(df1, df2)

- 没有指明要用哪个列进行连接，merge就会将重叠列的列名当做键，缺省做'内连接'。
- 内连接：使用两个表都有的key

左表df1

	key	data1
0	b	0
1	b	1
2	a	2
3	c	3
4	a	4
5	a	5
6	b	6

右表df2

	key	data2
0	a	0
1	b	1
2	b	2
3	d	3

合并后的表df3

	key	data1	data2
0	b	0	1
1	b	0	2
2	b	1	1
3	b	1	2
4	b	6	1
5	b	6	2
6	a	2	0
7	a	4	0
8	a	5	0

- 多对多连接产生的是**行的笛卡尔积**

雨课堂 Rain Classroom

主键合并数据

主键合并 —— merge函数

pd.merge(df1, df2, on='key', how='outer')

- 最好指明要用哪个列进行连接，并给出连接方式，如'外连接'。
- 外连接：使用两个表中所有的键

左表df1

	key	data1
0	b	0
1	b	1
2	a	2
3	c	3
4	a	4
5	a	5
6	b	6

右表df2

	key	data2
0	a	0
1	b	1
2	b	2
3	d	3

合并后的表df3

	key	data1	data2
0	b	0.0	1.0
1	b	0.0	2.0
2	b	1.0	1.0
3	b	1.0	2.0
4	b	6.0	1.0
5	b	6.0	2.0
6	a	2.0	0.0
7	a	4.0	0.0
8	a	5.0	0.0
9	c	3.0	NaN
10	d	NaN	3.0

主键合并数据

主键合并 —— merge函数

```
pd.merge(df1, df2, left_on='keyL', right_on='keyR', how='left')
```

- 如果两个对象的主键列名不同，可以分别指定列进行连接。
- 左连接：使用左表中所有的键

左表df1

	keyL	data1
0	b	0
1	b	1
2	a	2
3	c	3
4	a	4
5	b	5

右表df2

	keyR	data2
0	a	0
1	b	1
2	a	2
3	b	3
4	d	4

合并后的表df3

	keyL	data1	keyR	data2
0	b	0	b	1.0
1	b	0	b	3.0
2	b	1	b	1.0
3	b	1	b	3.0
4	a	2	a	0.0
5	a	2	a	2.0
6	c	3	NaN	NaN
7	a	4	a	0.0
8	a	4	a	2.0
9	b	5	b	1.0
10	b	5	b	3.0

47

主键合并数据

主键合并 —— merge函数

pd.merge(df1, df2, left_on='keyL', right_on='keyR', how='right')

- 如果两个对象的主键列名不同，可以分别指定列进行连接，并给出连接方式。
- 右连接：使用右表中所有的键

左表df1

	keyL	data1
0	b	0
1	b	1
2	a	2
3	c	3
4	a	4
5	b	5

右表df2

	keyR	data2
0	a	0
1	b	1
2	a	2
3	b	3
4	d	4

合并后的表df3

	keyL	data1	keyR	data2
0	b	0.0	b	1
1	b	1.0	b	1
2	b	5.0	b	1
3	b	0.0	b	3
4	b	1.0	b	3
5	b	5.0	b	3
6	a	2.0	a	0
7	a	4.0	a	0
8	a	2.0	a	2
9	a	4.0	a	2
10	NaN	NaN	d	4

148

主键合并数据

主键合并 —— merge函数

pd.merge(df1, df2, on=['key1', key2], how='outer')

- 根据多个键进行合并，传入一个由列名组成的列表即可。
  - 可以想象成把多个键形成元组，当成单个键进行连接（虽然实际上不是如此）

左表df1

	key1	key2	data1
0	foo	one	1
1	foo	two	2
2	bar	one	3

右表df2

	key1	key2	data2
0	foo	one	4
1	foo	one	5
2	bar	one	6
3	bar	two	7

合并后的表df3

	key1	key2	data1	data2
0	foo	one	1.0	4.0
1	foo	one	1.0	5.0
2	foo	two	2.0	NaN
3	bar	one	3.0	6.0
4	bar	two	NaN	7.0

雨课堂 Rain Classroom

主键合并数据

主键合并 —— merge函数

pd.merge(df1, df2, on='key' , suffixes = ('_left','_right'))

- 当合并的时候，左右表有重复列名，使用suffixes选项，在重叠列名上加后缀字符串。

左表df1

	key	k2	data1
0	foo	one	1
1	foo	two	2
2	bar	one	3

右表df2

	key	k2	data2
0	foo	one	4
1	foo	one	5
2	bar	one	6
3	bar	two	7

合并后的表df3

	key	k2_left	data1	k2_right	data2
0	foo	one	1	one	4
1	foo	one	1	one	5
2	foo	two	2	one	4
3	foo	two	2	one	5
4	bar	one	3	one	6
5	bar	one	3	two	7

雨课堂 Rain Classroom

主键合并数据（自我练习）

主键合并 —— merge函数

```
order = pd.read_csv('data/meal_order_info.csv', sep=',', encoding='gb18030')
order['info_id'] = order['info_id'].astype('str') #info_id转换为字符串格式，为合并做准备
# 订单详情表和订单信息表都有订单编号
# 在订单详情表中为order_id，在订单信息表中为info_id
order_detail = pd.merge(detail1,order, left_on='order_id',right_on = 'info_id')
print('detail1订单详情表的原始形状为：',detail1.shape)
print('order订单信息表的原始形状为：',order.shape)
print('订单详情表和订单信息表主键合并后的形状为：',order_detail.shape)
```

```
detail1订单详情表的原始形状为： (2779, 19)
order订单信息表的原始形状为： (945, 21)
订单详情表和订单信息表主键合并后的形状为： (2779, 40)
```

重叠合并数据

combine_first方法

- 数据分析和处理过程中偶尔会出现两份数据的内容几乎一致的情况，但是某些特征在其中一张表上是完整的，而在另外一张表上的数据则是缺失的。
- 这时可以采用将数据一对一比较，然后进行填充。除此之外，可以用combine_first方法进行重叠数据合并，其原理如下。

表8

	0	1	2
0	NaN	3.0	5.0
1	NaN	4.6	NaN
2	NaN	7.0	NaN

表9

	0	1	2
1	42	NaN	8.2
2	10	7.0	4.0

合并后表10

	0	1	2
0	NaN	3.0	5.0
1	42	4.6	8.2
2	10	7.0	4.0

雨课堂 Rain Classroom

重叠合并数据

combine_first方法

- 重叠合并在其他工具或者语言中并不常见，但是pandas库的开发者希望pandas能够解决几乎所有的数据分析问题，因此提供了combine_first方法来进行重叠数据合并。
- combine_first的具体用法：*pandas.DataFrame.combine_first(other)*
- 参数及其说明如下。

参数名称	说明
other	接收DataFrame。表示参与重叠合并的另一个DataFrame。无默认。

雨课堂 Rain Classroom

重叠合并数据

combine_first方法

df1.combine_first(df2)

- combine_first可以被看做，用传递对象中的数据为调用对象的缺失数据“打补丁”

左表df1

	a	b	c
0	1.0	NaN	2
1	NaN	2.0	6
2	5.0	NaN	10
3	NaN	6.0	14

右表df2

	a	b
0	5.0	NaN
1	4.0	3.0
2	NaN	4.0
3	3.0	6.0
4	7.0	8.0

合并后的表df3

	a	b	c
0	1.0	NaN	2.0
1	4.0	2.0	6.0
2	5.0	4.0	10.0
3	3.0	6.0	14.0
4	7.0	8.0	NaN

重叠合并数据（自我练习）

combine_first方法

```
##建立两个字典，除了ID外，别的特征互补
dict1 = {'ID':[1,2,3,4,5,6,7,8,9],
         'System':['win10','win10',np.nan,'win10',np.nan,np.nan,'win7','win7','win8'],
         'cpu':['i7','i5',np.nan,'i7',np.nan,np.nan,'i5','i5','i3']}
dict2 = {'ID':[1,2,3,4,5,6,7,8,9],
         'System':[np.nan,np.nan,'win7',np.nan,'win8','win7',np.nan,np.nan,np.nan],
         'cpu':[np.nan,np.nan,'i3',np.nan,'i7  ','i5',np.nan,np.nan,np.nan]}
## 转换两个字典为DataFrame
df5 = pd.DataFrame(dict1)
df6 = pd.DataFrame(dict2)
print('经过重叠合并后的数据为: \n',df5.combine_first(df6))
```

155

雨课堂 Rain Classroom

重叠合并数据（自我练习）

combine_first方法

```
##建立两个字典，除了ID外，别的特征互补
dict1 = {'ID':[1,2,3,4,5,6,7,8,9],
        'System':['win10','win10',np.nan,'win10',np.n                    3'],
        'cpu':['i7','i5',np.nan,'i7',np.nan,np.nan,'i5','i5
dict2 = {'ID':[1,2,3,4,5,6,7,8,9],
        'System':[np.nan,np.nan,'win7',np.nan,'win8',                    ],
        'cpu':[np.nan,np.nan,'i3',np.nan,'i7  ','i5',np.n
## 转换两个字典为DataFrame
df5 = pd.DataFrame(dict1)
df6 = pd.DataFrame(dict2)
print('经过重叠合并后的数据为: \n',df5.combine_first(df6))
```

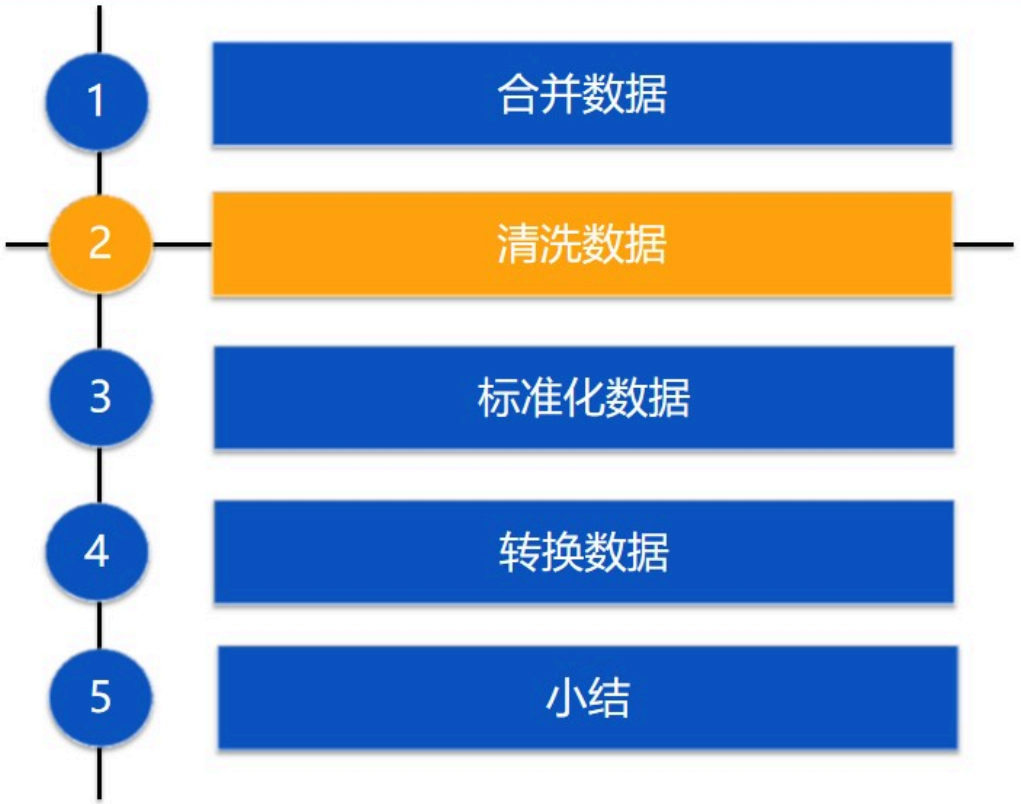
经过重叠合并后的数据为:

	ID	System	cpu
0	1	win10	i7
1	2	win10	i5
2	3	win7	i3
3	4	win10	i7
4	5	win8	i7
5	6	win7	i5
6	7	win7	i5
7	8	win7	i5
8	9	win8	i3

156

雨课堂 Rain Classroom

目录



清洗数据

任务描述

- **数据重复**会导致数据的方差变小，数据分布发生较大变化。
- **数据缺失**会导致样本信息减少，不仅增加了数据分析的难度，而且会导致数据分析的结果产生偏差。
- **异常值**则会产生“伪回归”。
- 因此需要对数据进行检测，查询是否有重复值、缺失值和异常值，并且要对这些数据进行适当的处理。

任务分析

(1) 检测与处理数据的重复值。

(2) 检测与处理数据的缺失值。

(3) 检测与处理数据的异常值

❶检测与处理重复值

- 常见的数据重复分为两种：
  - 一种为**记录重复（行相同）**， 即一个或者多个特征的某几条记录的值完全相同；
  - 另一种为**特征重复（列相同）**，即存在一个或者多个特征名称不同，但数据完全相同的情况。

雨课堂 Rain Classroom

❶检测与处理重复值

1. 记录重复：即一个或者多个特征某几个记录的值完全相同

- 菜品订单详情表中的 dishes_name 特征存放了每个订单的菜品。要找出所有已点菜品，最简单的方法就是利用去重操作实现。
- **方法一：是利用列表（list）去重，自定义去重函数：**

```
def delRep(list1):
    list2=[]
    for i in list1:
        if i not in list2:
            list2.append(i)
    return list2
```


雨课堂 Rain Classroom

❶检测与处理重复值

1. 记录重复：即一个或者多个特征某几个记录的值完全相同

- 方法一：是利用列表（list）去重，自定义去重函数：

```
detail = pd.read_csv('data/detail.csv', index_col=0, encoding = 'gbk')
dishes=list(detail['dishes_name'])          #将dishes_name从DataFrame中提取出来
print('去重前菜品总数为：', len(dishes))

dish = delRep(dishes)       #使用自定义的去重函数去重
print('方法一去重后菜品总数为：', len(dish))
```

```
去重前菜品总数为： 10037
方法一去重后菜品总数为： 145
```

```
##自定义去重函数
def delRep(list1):
    list2=[]
    for i in list1:
        if i not in list2:
            list2.append(i)
    return list2
```


雨课堂 Rain Classroom

❶检测与处理重复值

1. 记录重复：即一个或者多个特征某几个记录的值完全相同

- 方法二是利用集合（set）的元素是唯一的特性去重

```
detail = pd.read_csv('data/detail.csv', index_col=0, encoding = 'gbk')
dishes=list(detail['dishes_name'])          #将dishes_name从DataFrame中提取出来
print('去重前菜品总数为：', len(dishes))

dish_set = set(dishes)          #利用set的特性去重
print('方法二去重后菜品总数为：', len(dish_set))
```

```
去重前菜品总数为： 10037
方法二去重后菜品总数为： 145
```

❶检测与处理重复值

1. 记录重复

- 比较上述两种方法可以发现，方法一代码冗长，会拖慢整个数据分析的进度。
- 方法二使用了集合元素的唯一性，看似代码简单了许多，但会导致数据的排列发生改变。
- 不同方法去重前后的部分数据排列比较：

源数据	方法一去重后数据	方法二去重后数据
蒜蓉生蚝	蒜蓉生蚝	纸杯蛋糕
蒙古烤羊腿	蒙古烤羊腿	爆炒鳝碌
大蒜苋菜	大蒜苋菜	黄尾袋鼠西拉子红葡萄酒
芝麻烤紫菜	芝麻烤紫菜	白斩鸡
蒜香包	蒜香包	香菇鸡肉粥
白斩鸡	白斩鸡	农夫山泉NFC果汁100%橙汁

❶检测与处理重复值

1. 记录重复 —— drop_duplicates( )方法去重

- pandas的drop_duplicates方法只对DataFrame或者Series类型有效。
- 它不会改变数据原始排列，并且兼具代码简洁和运行稳定的特点。
- 不仅支持单一特征的数据去重，还能够依据DataFrame的几个特征(subset设置)进行去重操作。

DataFrame(Series).drop_duplicates(self, subset=None, keep='first', inplace=False)

雨课堂 Rain Classroom

❶检测与处理重复值

1. 记录重复 —— drop_duplicates( )方法去重

- 对菜品订单详情表中的 dishes_name 列利用 drop_duplicates 方法进行去重操作

```
dishes_name = detail['dishes_name'].drop_duplicates()
print('drop_duplicates方法去重之后菜品总数为：',len(dishes_name))
```

```
drop_duplicates方法去重之后菜品总数为：  145
```


雨课堂 Rain Classroom

❶检测与处理重复值

1. 记录重复 —— drop_duplicates( )方法去重

- drop_duplicates 方法不仅支持单一特征的数据去重，还能够依据几个特征进行去重操作，如下：

```
print('去重之前订单详情表的形状为：', detail.shape)
shapeDet = detail.drop_duplicates(subset = ['order_id','emp_id']).shape
print('依照订单编号，会员编号去重之后订单详情表大小为：', shapeDet)
```

```
去重之前订单详情表的形状为： (10037, 18)
依照订单编号，会员编号去重之后订单详情表大小为: (942, 18)
```

❶检测与处理重复值

2. 特征重复 —— 利用特征间的相似度去重（数值型特征）

- 结合相关的数学和统计学知识，去除连续型特征重复可以利用特征间的相似度将两个相似度为1的特征去除一个。（降维）
- **在pandas中相似度的计算方法为 corr （即，correlation缩写）**
  - 使用该方法计算相似度时，有三种方法：
    - “pearson”法（**默认方法**）；“spearman”法；“kendall”法
  - 可以通过“method”参数调节使用的方法
- 通常情况下通过以下取值范围判断变量的相关强度：

0.8-1.0 极强相关；　　0.6-0.8 强相关　　0.4-0.6 中等程度相关

0.2-0.4 弱相关；　　0.0-0.2 极弱相关或无相关

❶检测与处理重复值——实例

2. 特征重复 —— 利用特征间的相似度去重（数值型特征）

- 使用 “kendall” 法求出菜品订单详情表（midetail1.xlsx）数据中counts列和amounts列的相似度矩阵。

```
# 求取销量和售价的相似度
corrDet = detail[['counts','amounts']].corr(method='kendall')
print('销量和售价的kendall相似度为：\n', corrDet)
```

```
销量和售价的kendall相似度为：
             counts    amounts
counts    1.000000 -0.229968
amounts -0.229968  1.000000
```

168

❶检测与处理重复值

2. 特征重复 —— 利用特征间的相似度去重（数值型特征）

- 但是通过相似度矩阵去重存在一个弊端，该方法只能对数值型重复特征去重，类别型特征之间无法通过计算相似系数来衡量相似度，因此无法根据相似度矩阵对其进行去重处理。
- 对订单详情表中的**dishes_name**、counts和amounts这3个特征进行pearson法相似度矩阵的求解，但最终却只能得到counts和amounts特征2*2的相似度矩阵。

```
In [7]: corrDet1 = detail[['dishes_name','counts','amounts']].corr(method='pearson')
   ...: print('菜品名称，销量和售价的pearson相似度为: \n',corrDet1)
菜品名称，销量和售价的pearson相似度为:
             counts   amounts
counts    1.000000 -0.159264
amounts -0.159264  1.000000
```


雨课堂 Rain Classroom

❶检测与处理重复值

2. 特征重复 —— 利用DataFrame.equals方法

```
#定义求取特征是否完全相同的矩阵的函数
def FeatureEquals(df):
    dfEquals=pd.DataFrame([],columns=df.columns,index=df.columns)    #构建一个空DataFrame
    for i in df.columns:
        for j in df.columns:
            dfEquals.loc[i, j]=df.loc[:, i].equals(df.loc[:, j])
    return dfEquals     #返回相似度矩阵，如果矩阵中i行j列的值为True，说明原数据的i列和j列重复
```

如果df数据中第i列与第j列完全相同，
则相似度矩阵dfEquals[i,j]的值为True

```
# 应用上述函数
detEquals=FeatureEquals(detail)
print('detail的特征相等矩阵的前5行5列为：\n', detEquals.iloc[:5, :5])
```

❶检测与处理重复值

2. 特征重复 —— 利用DataFrame.equals方法

```
In [9]: detEquals=FeatureEquals(detail)
   ...: print('detail的特征相等矩阵的前5行5列为: \n',detEquals.iloc[:5,:5])
detail的特征相等矩阵的前5行5列为:
                    order_id     ...       dishes_name
order_id               True      ...             False
dishes_id             False      ...             False
logicprn_name         False      ...             False
parent_class_name     False      ...             False
dishes_name           False      ...              True

[5 rows x 5 columns]
```


雨课堂 Rain Classroom

❶检测与处理重复值

2. 特征重复 —— 利用DataFrame.equals方法

- 再通过遍历的方式筛选出完全重复的特征

```
#遍历所有数据
lenDet = detEquals.shape[0]
dupCol = []
for k in range(lenDet):
    for l in range(k+1, lenDet):
        if detEquals.iloc[k,l] & (detEquals.columns[l] not in dupCol):
            dupCol.append(detEquals.columns[l])
#进行去重操作
print('需要删除的列为：',dupCol)
detail.drop(dupCol, axis=1, inplace=True)
print('删除多余列后detail的特征数目为：', detail.shape[1])
```

df数据中k列和l列相同，且l列不在去重数组中

所有待去重的columns

雨课堂 Rain Classroom

❶检测与处理重复值

2. 特征重复 —— 利用DataFrame.equals方法

- 执行结果如下：

```
In [12]: lenDet = detEquals.shape[0]
    ...: dupCol = []
    ...: for k in range(lenDet):
    ...:     for l in range(k+1,lenDet):
    ...:         if detEquals.iloc[k,l] & (detEquals.columns[l] not in dupCol):
    ...:             dupCol.append(detEquals.columns[l])
    ...:
    ...: ##进行去重操作
    ...: print('需要删除的列为：',dupCol)
    ...: detail.drop(dupCol,axis=1,inplace=True)
    ...: print('删除多余列后detail的特征数目为：',detail.shape[1])
需要删除的列为： ['parent_class_name', 'cost', 'discount_amt', 'discount_reason',
'kick_back', 'add_info', 'bar_code', 'add_inprice']
删除多余列后detail的特征数目为： 10
```

❷检测与处理缺失值

Why + How

- 数据中的某个或某些特征的值是不完整的，这些值称为**缺失值**。
- pandas提供了**识别缺失值的方法 isnull** 以及识别非缺失值的方法 **notnull**，这两种方法在使用时返回的都是布尔值 True 和 False。
- 结合sum函数和 isnull、notnull 函数，可以检测数据中缺失值的分布以及数据中一共含有多少缺失值。
- Isnull 和 notnull 之间结果正好相反，因此使用其中任意一个都可以判断出数据中缺失值的位置。

❷检测与处理缺失值

利用 isnull 或 notnull 找到缺失值

```
print('detail每个特征缺失的数目为：\n', detail.isnull().sum())
print('detail每个特征非缺失的数目为：\n', detail.notnull().sum())
```

```
detail每个特征缺失的数目为：
 order_id              0
dishes_id              0
logicprn_name      10037
dishes_name            0
itemis_add             0
counts                 0
amounts                0
place_order_time       0
picture_file           0
emp_id                 0
dtype: int64
```

```
detail每个特征非缺失的数目为：
 order_id           10037
dishes_id           10037
logicprn_name           0
dishes_name         10037
itemis_add          10037
counts              10037
amounts             10037
place_order_time    10037
picture_file        10037
emp_id              10037
dtype: int64
```

❷检测与处理缺失值

1. 删除法：将含有缺失值的特征或者记录删除

- 删除法分为 删除记录(行) 和 删除特征(列) 两种。
- 它属于利用减少样本量来换取信息完整度的一种方法，是一种最简单的缺失值处理方法。

- pandas中提供了简便的删除缺失值的方法**dropna**，该方法既可以删除记录(axis=0，行)，亦可以删除特征(axis=1，列)。

pandas.DataFrame.dropna(self, axis=0, how='any', thresh=None, subset=None, inplace=False)

❷检测与处理缺失值（自我练习）

1. 删除法：将含有缺失值的特征或者记录删除

```
print('去除缺失的列前detail的形状为：', detail.shape)
print('去除缺失的列后detail的形状为：', detail.dropna(axis = 1,how ='any').shape )
```

```
去除缺失的列前detail的形状为：  (10037, 10)
去除缺失的列后detail的形状为：  (10037, 9)
```

- 当 how 参数取值为 any 时（默认值），只要特征存在缺失值，就删除这个特征。
- 若 how 参数取 all，则整个特征全部为缺失值时才会执行删除操作。

雨课堂 Rain Classroom

❷检测与处理缺失值

2. 替换法

- 替换法是指用一个特定的值替换缺失值。
- 特征可分为数值型和类别型，两者出现缺失值时的处理方法也是不同的。
  - 缺失值所在特征为**数值型**时，通常利用其均值、中位数和众数等描述其集中趋势的统计量来代替缺失值。
  - 缺失值所在特征为**类别型**时，则选择使用众数来替换缺失值。
- pandas库中提供了缺失值替换的方法：**fillna**，其基本语法如下：

```
pandas.DataFrame.fillna(value=None, method=None, axis=None, inplace=False,
                        limit=None)
```

❷检测与处理缺失值

2. 替换法 —— 实例

```
detail = detail.fillna(-99)
print('detail每个特征缺失的数目为：\n',detail.isnull().sum())
```

```
detail每个特征缺失的数目为:
 order_id             0
dishes_id             0
logicprn_name         0
dishes_name           0
itemis_add            0
counts                0
amounts               0
place_order_time      0
picture_file          0
emp_id                0
dtype: int64
```

❷检测与处理缺失值

3. 插值法

- 删除法简单易行，但是会引起数据结构变动，样本减少；
- 替换法使用难度较低，但是会影响数据的标准差，导致信息量变动。
- 在面对数据缺失问题时，除了这两种方法之外，还有一种常用的方法—插值法。
- 常用的插值法有**一维插值、多项式插值和样条插值**等。
- Pandas提供了对应的名为interploate的插值方法，能够进行上述部分插值操作，但是SciPy的interploate模块更加全面。

雨课堂 Rain Classroom

❷检测与处理缺失值

3. 插值法

- 常用的插值法有一维插值、多项式插值和样条插值等：
  - 一维插值是一种较为简单的插值方法，它针对已知的值求出线性方程，通过求解线性方程得到缺失值。
  - 多项式插值是利用已知的值拟合一个多项式，使得现有的数据满足这个多项式，再利用这个多项式求解缺失值，常见的多项式插值法有拉格朗日插值和牛顿插值等。
  - 样条插值是以可变样条来作出一条经过一系列点的光滑曲线的插值方法，插值样条由一些多项式组成，每一个多项式都是由相邻两个数据点决定，这样可以保证两个相邻多项式及其导数在连接处连续。

❷检测与处理缺失值

3. 插值法（一维插值）

- 当样本数据变化归因与一个独立变量，就是一维插值；当样本数据变化归因于多个独立变量，就是用多维插值
- 将内插类型指定为字符串 ‘linear’ **线性，也是默认一维插值，** ‘nearest’ **最近，** ‘zero’ **零，** ‘slinear’ **线性，** ‘quadratic’ **二次，** ‘cubic’ **三次，** ‘previous’ **上一个，** ‘next’ **下一个。**

参数	作用
‘zero’ 、 ‘nearest’	阶梯插值，相当于0阶B样条插值
‘slinear’ 、 ‘linear’	线性插值，用一条直线连接所有的取样点，相当于一阶B样条插值
‘quadratic’ 、 ‘cubic’	二阶和三阶B样条插值，更高阶的曲线可以直接使用整数值指定

❷检测与处理缺失值

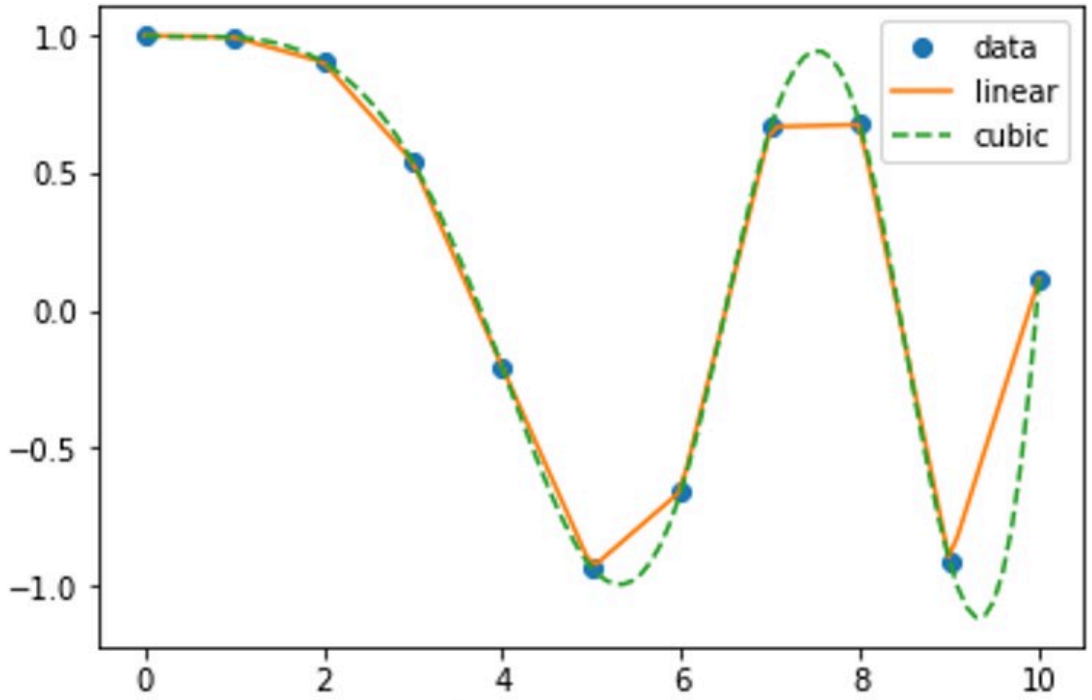
3. 插值法（一维插值）

```
from scipy.interpolate import interp1d      #Interpolate a 1-D function.
import matplotlib.pyplot as plt
import numpy as np
x = np.linspace(0, 10, num=11, endpoint=True)
y = np.cos(-x**2/9.0)
f = interp1d(x, y)                          #返回插值函数（默认是线性linear插值函数）
f2 = interp1d(x, y, kind='cubic')           #返回三次插值函数（cubic）
xnew = np.linspace(0, 10, num=100, endpoint=True)
plt.plot(x, y, 'o', xnew, f(xnew), '-', xnew, f2(xnew), '--')
plt.legend(['data', 'linear', 'cubic'], loc='best')
plt.show()
```


雨课堂 Rain Classroom

❷检测与处理缺失值

3. 插值法（一维插值）



❷检测与处理缺失值（自我练习）

3. 插值法（一维插值）

```
import matplotlib.pyplot as plt
import numpy as np
from scipy.interpolate import interp1d                    #一维数据插值函数

x = np.array([1,2,3,4,5,8,9,10])                          #创建自变量x
y1=np.array([2,8,18,32,50,128,162,200])                   #创建因变量y1
y2=np.array([3,5,7,9,11,17,19,21])                        #创建因变量y2

LinearIns1 = interp1d(x, y1, kind='linear')               #返回线性插值函数，拟合x,y1
LinearIns2 = interp1d(x, y2, kind='linear')               #返回线性插值函数，拟合x,y2

plt.plot(x, y1 ,'.', x , y2,'*')                          #绘制原始数据
xinsert = np.array([6,7])
y1insert = LinearIns1([6,7])                              #根据（x,y1）生成插值点
y2insert = LinearIns2([6,7])                              #根据（x,y2）生成插值点
```

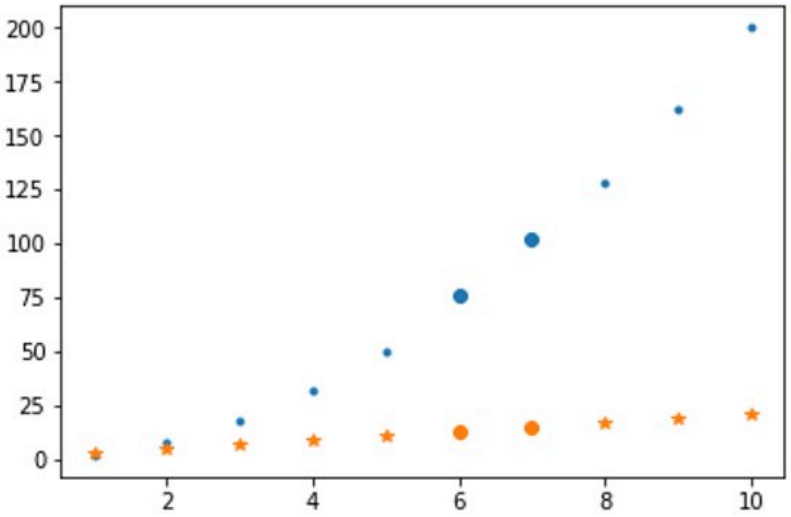

雨课堂 Rain Classroom

❷检测与处理缺失值（自我练习）

3. 插值法（一维插值）—— 查看两个差值点

```
plt.scatter(xinsert, y1insert)                    #绘制插值点
plt.scatter(xinsert, y2insert)                    #绘制插值点

print('当x为6、7时，使用线性插值y1为：', LinearIns1([6,7]))      #输出插值数据
print('当x为6、7时，使用线性插值y2为：', LinearIns2([6,7]))      #输出插值数据
```




```
    ...: print('当x为6、7时，使用线性插值y1为：',LinearInsValue1([6,7]))
    ...: print('当x为6、7时，使用线性插值y2为：',LinearInsValue2([6,7]))
当x为6、7时，使用线性插值y1为：  [ 76. 102.]
当x为6、7时，使用线性插值y2为：  [13. 15.]
```

❷检测与处理缺失值

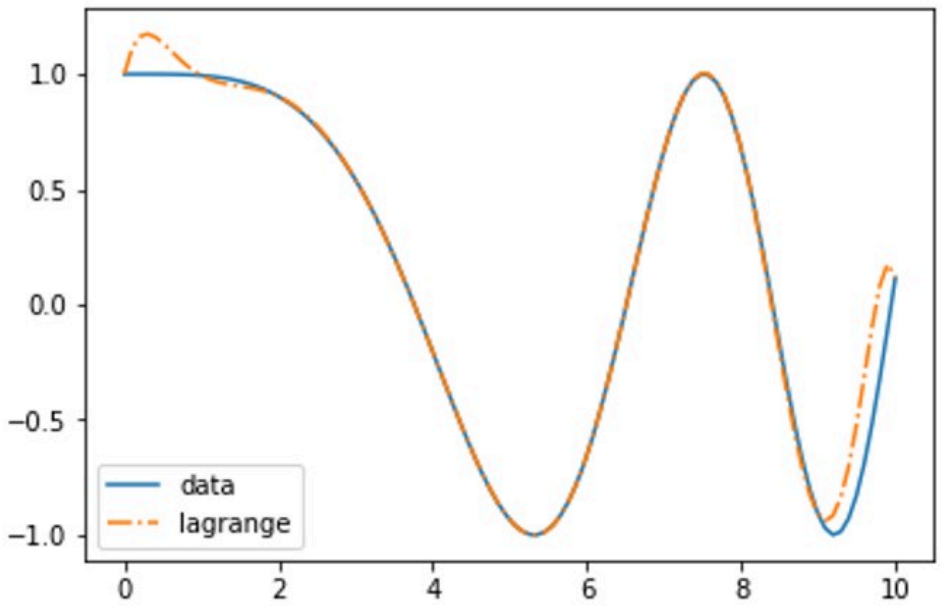
3. 插值法（拉格朗日插值）

```
from scipy.interpolate import lagrange
import matplotlib.pyplot as plt
import numpy as np
x = np.linspace(0, 10, num=11, endpoint=True)
y = np.cos(-x**2/9.0)
f3 = lagrange(x,y)
xnew = np.linspace(0, 10, num=100, endpoint=True)
plt.plot(xnew, np.cos(-xnew**2/9.0), '-', xnew, f3(xnew), '-.')
plt.legend(['data','lagrange'], loc='best')
plt.show()
```

187

❷检测与处理缺失值

3. 插值法（拉格朗日插值）



拉格朗日插值在数值上不稳定。 即使选择它们，也不要期望使用超过20个点。

Lagrange插值多项式的缺点：

- **插值基函数计算复杂**
- **当增加一个新点时需重新计算**
- **插值多项式从n次增加到n+1次需重新计算**
- **插值曲线在节点处有尖点，不光滑，节点处不可导**
- **高次插值的精度不一定高**

雨课堂 Rain Classroom

❷检测与处理缺失值（自我练习）

3. 插值法（拉格朗日插值）—— 查看两个差值点

```
from scipy.interpolate import lagrange      #拉格朗日插值
LargeInsValue1 = lagrange(x, y1)            #拉格朗日插值拟合x,y1
LargeInsValue2 = lagrange(x, y2)            #拉格朗日插值拟合x,y2
print('当x为6,7时，使用拉格朗日插值y1为：', LargeInsValue1([6,7]))
print('当x为6,7时，使用拉格朗日插值y2为：', LargeInsValue2([6,7]))
```

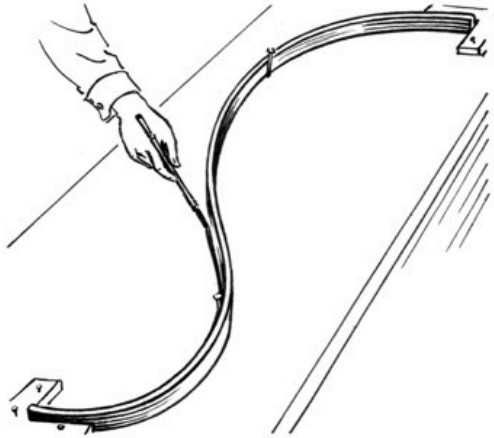
```
当x为6,7时，使用拉格朗日插值y1为： [72. 98.]
当x为6,7时，使用拉格朗日插值y2为： [13. 15.]
```


雨课堂 Rain Classroom

❷检测与处理缺失值

3. 插值法（样条插值）

- 样条来源于早期工程制图，为了将一些固定点连成一条光滑的曲线，采用具有弹性的木条固定在这些点上，如图所示。通过样条画出来的曲线不仅经过各固定点，而且连续光滑。后来这一技术发展成了数学工具——样条函数，在很多领域都有应用，样条插值就是典型的应用场景。
- 样条函数，顾名思义就是描述图中那个木条的函数，在数学上把它定义成一个分段多项式函数，以三次多项式为例：



$$f(x)=\begin{cases} a_1x^3+b_1x^2+c_1x+d_1 & \text{if } x\in[x_1,x_2] \\ a_2x^3+b_2x^2+c_2x+d_2 & \text{if } x\in(x_2,x_3] \\ \cdots & \\ a_nx^3+b_nx^2+c_nx+d_n & \text{if } x\in(x_n,x_{n+1}]\ . \end{cases}$$

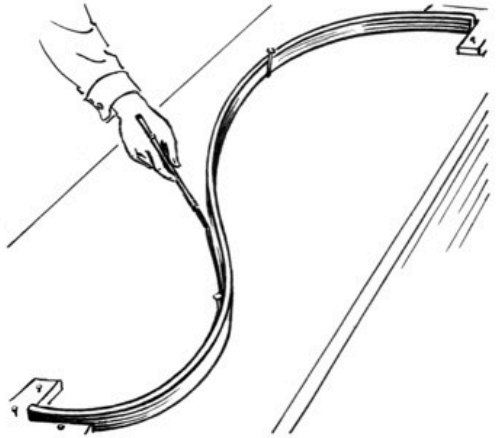
雨课堂 Rain Classroom

❷检测与处理缺失值

3. 插值法（样条插值）

- 样条函数，顾名思义就是描述图中那个木条的函数，在数学上把它定义成一个分段多项式函数，以三次多项式为例：

$$f(x)=\begin{cases}a_1x^3+b_1x^2+c_1x+d_1 & \text{if } x\in[x_1,x_2]\\ a_2x^3+b_2x^2+c_2x+d_2 & \text{if } x\in(x_2,x_3]\\ \cdots & \\ a_nx^3+b_nx^2+c_nx+d_n & \text{if } x\in(x_n,x_{n+1}]\ .\end{cases}$$



- 也就是说，每两个点之间用一个多项式来表示，这些多项式的阶数相同，但是系数是不一样的。样条插值的目的就是求这些多项式系数，已知系数，那些中间点的值就能唯一确定了。不难想象，随着多项式阶数的提升，端点处会更加平滑，但计算量也越大。

雨课堂 Rain Classroom

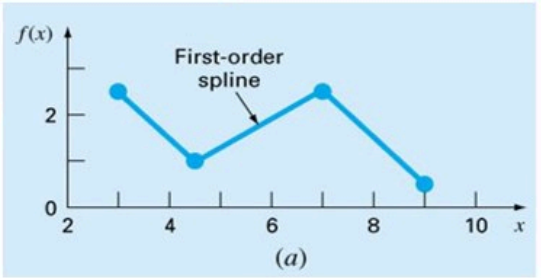
❷检测与处理缺失值

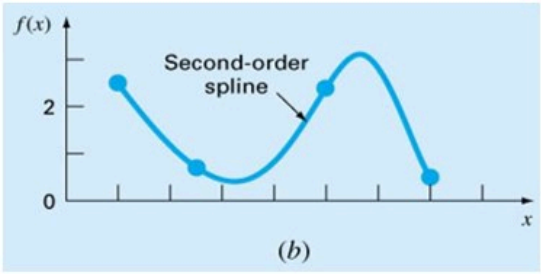
3. 插值法（样条插值）

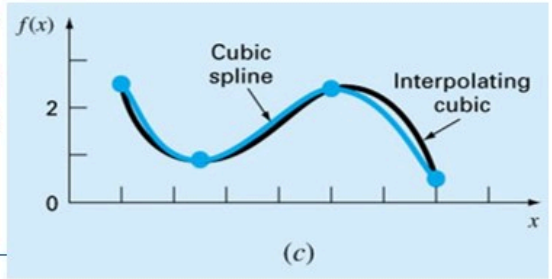
- 样条函数，顾名思义就是描述图中那个木条的函数，在数学上把它定义成一个分段多项式函数，以三次多项式为例：

$$f(x)=\begin{cases} a_1x^3+b_1x^2+c_1x+d_1 & \text{if } x\in[x_1,x_2] \\ a_2x^3+b_2x^2+c_2x+d_2 & \text{if } x\in(x_2,x_3] \\ \cdots \\ a_nx^3+b_nx^2+c_nx+d_n & \text{if } x\in(x_n,x_{n+1}]\ . \end{cases}$$

- 也就是说，每两个点之间用一个多项式来表示，这些多项式的阶数相同，但是系数是不一样的。样条插值的目的就是求这些多项式系数，已知系数，那些中间点的值就能唯一确定了。不难想象，随着多项式阶数的提升，端点处会更加平滑，但计算量也越大。







❷检测与处理缺失值

3. 插值法（样条插值）

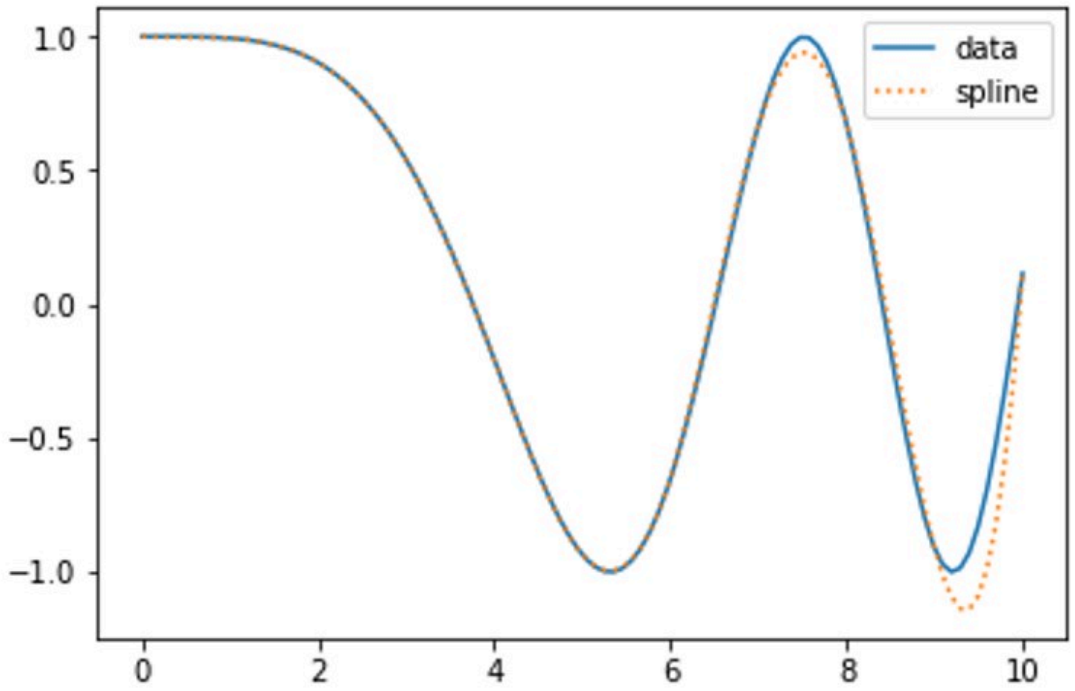
```
from scipy.interpolate import spline      #三次样条插值
import matplotlib.pyplot as plt
import numpy as np
x = np.linspace(0, 10, num=11, endpoint=True)
y = np.cos(-x**2/9.0)
xnew = np.linspace(0, 10, num=100, endpoint=True)
f4values = spline(x,y,xnew)                    #x, y 数据定义了曲线，使用样条差值法，依据
                                               #曲线通过xnew值估算 y’ = f4values
plt.plot(xnew, np.cos(-xnew**2/9.0), '-', xnew, f4values, ':')
plt.legend(['data', 'spline'], loc='best')
plt.show()
```

193

雨课堂 Rain Classroom

❷检测与处理缺失值

3. 插值法（样条插值）



❷检测与处理缺失值（自我练习）

3. 插值法（样条插值）—— 查看两个差值点

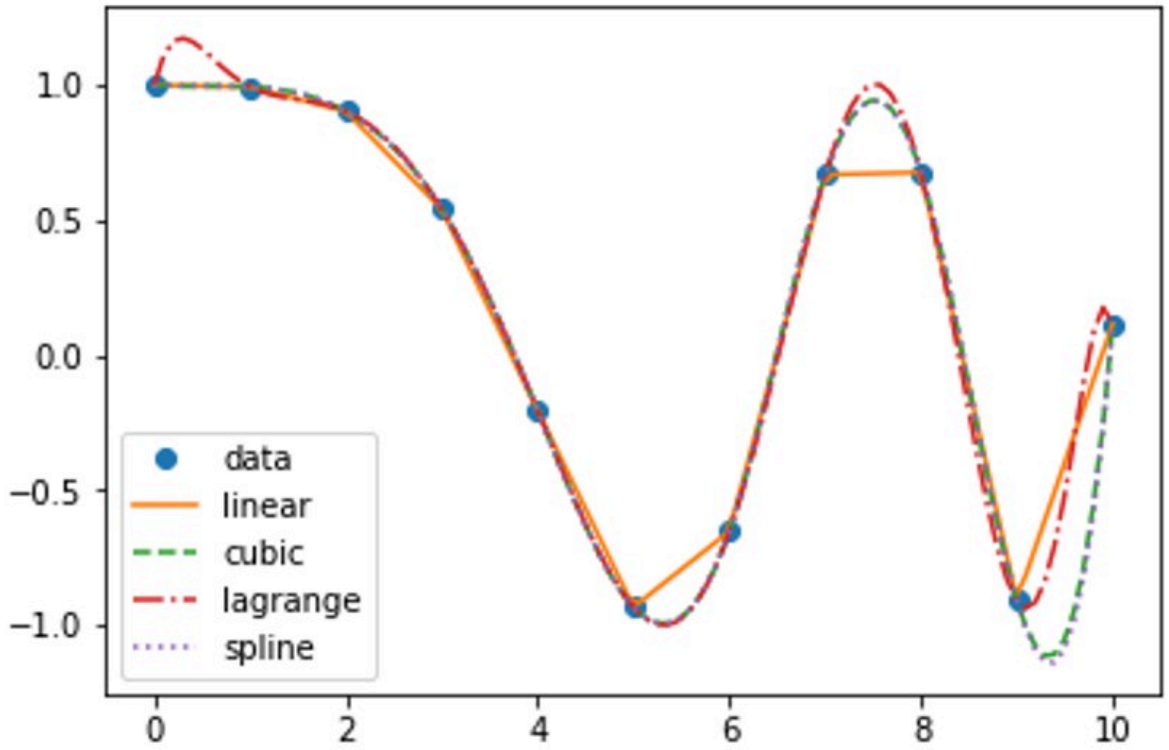
```
from scipy.interpolate import spline
SplineInsValue1 = spline(x,y1, xnew=np.array([6,7]))      #样条插值拟合x,y1
SplineInsValue2 = spline(x,y2, xnew=np.array([6,7]))      #样条插值拟合x,y2
print('当x为6,7时，使用样条插值y1为：', SplineInsValue1)
print('当x为6,7时，使用样条插值y2为：', SplineInsValue2)
```

```
当x为6,7时，使用样条插值y1为： [72. 98.]
当x为6,7时，使用样条插值y2为： [13. 15.]
```


雨课堂 Rain Classroom

❷检测与处理缺失值

3. 插值法（综合）



196

❷检测与处理缺失值

3. 插值法 —— 总结

- 从拟合结果可以看出多项式插值和样条插值在两种情况下拟合都非常出色，线性插值法只在自变量和因变量为线性关系的情况下拟合才较为出色。
- 而在实际分析过程中，自变量与因变量的关系是线性的情况非常少见，所以在大多数情况下，多项式插值和样条插值是较为合适的选择。
- SciPy库中的interpolate模块除了提供常规的插值法外，还提供了例如在图形学领域具有重要作用的重心坐标插值（BarycentricInterpolator）等。在实际应用中，需要根据不同的场景，选择合适的插值方法。

❸检测与处理异常值

异常值

- 异常值是指数据中个别值的数值明显偏离其余的数值，有时也称为离群点。
- 检测异常值就是检验数据中是否有录入错误以及是否含有不合理的数据。
- 异常值的存在对数据分析十分危险，如果计算分析过程的数据有异常值，那么会对结果会产生不良影响，从而导致分析结果产生偏差乃至错误。
- 常用的异常值检测主要为 **3σ原则** 和 **箱线图分析** 两种方法。

雨课堂 Rain Classroom

❸检测与处理异常值

1. 3σ原则检测异常值

- 3σ原则又称为拉依达法则。该法则就是先假设一组检测数据只含有随机误差，对原始数据进行计算处理得到标准差，然后按一定的概率确定一个区间，认为误差超过这个区间的就属于异常值。

- 这种判别处理方法仅适用于对正态或近似正态分布的样本数据进行处理，如表所示，其中σ代表标准差，μ代表均值，x=μ为图形的对称轴。

数值分布	在数据中的占比
$(\mu-\sigma,\mu+\sigma)$	0.6827
$(\mu-2\sigma,\mu+2\sigma)$	0.9545
$(\mu-3\sigma,\mu+3\sigma)$	0.9973

- 通过表格内容看出，数据的数值分布几乎全部集中在区间(μ-3σ,μ+3σ)内，超出这个范围的数据仅占不到0.3%。故根据小概率原理，可以认为超出3σ的部分数据为异常数据。

雨课堂 Rain Classroom

❸检测与处理异常值

1. 3σ原则检测异常值 —— 实例

➢ 自行构建**3σ函数**，进行异常值识别

```
import pandas as pd
detail = pd.read_csv('data/detail.csv', index_col=0, encoding = 'gbk')
def outRange(Ser1):
    boolInd = (Ser1<Ser1.mean()-3*Ser1.std()) | (Ser1>Ser1.mean()+3*Ser1. std()) #判断是否在区间外
    index = np.arange(Ser1.shape[0])[boolInd]          #布尔索引，返回区间之外数据的index
    outrange = Ser1.iloc[index]
    return outrange
outlier = outRange(detail['counts'])
print('使用拉依达准则判定异常值个数为:', outlier.shape[0])
print('异常值的最大值为：', outlier.max())
print('异常值的最小值为：', outlier.min())
```

❸检测与处理异常值

1. 3σ原则检测异常值 —— 实例

- 自行构建**3σ函数**，进行异常值识别

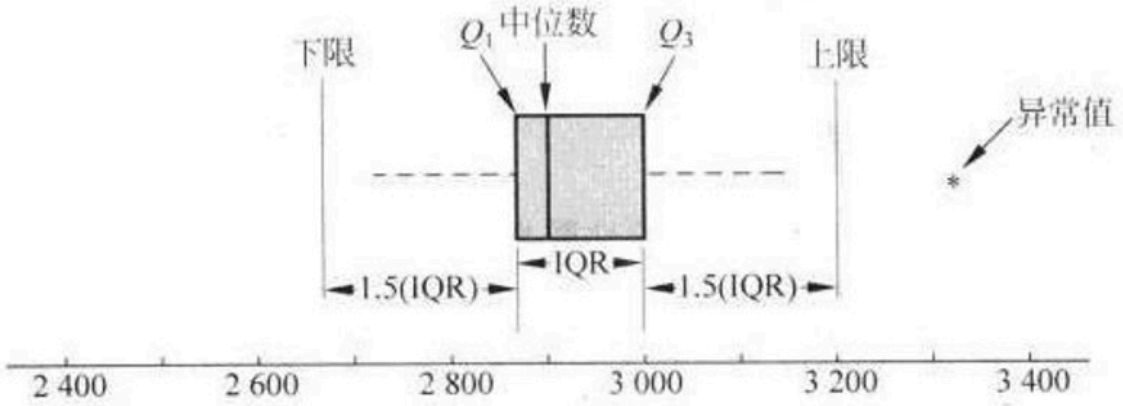
```
import pandas as pd
detail = pd.read_csv('data/detail.csv', index_col=0, encoding = 'gbk')
def outRange(Ser1):
    boolInd = (Ser1<Ser1.mean()-3*Ser1.std()) | (Ser1>Ser1.mean()+3*Ser1. std()) #判断是否在区间外
    index = np.arange(Ser1.shape[0])[boolInd]    #布尔索引，返回区间之外数据的index
    outrange = Ser1.iloc[index]                  #通过index取数据
    return outrange        #返回所有离群数据
outlier = outRange(detail['counts'])
print('使用拉依达准则判定异常值个数为:',outlier.shape[0])
print('异常值的最大值为：',outlier.max())
print('异常值的最小值为：',outlier.min())
```

```
使用拉依达准则判定异常值个数为： 209
异常值的最大值为：  10
异常值的最小值为：  3
```

❸检测与处理异常值

2. 箱线图分析

- 箱型图提供了识别异常值的一个标准：即小于**QL-1.5 IQR | 大于QU+1.5 IQR** 的值。
  - QL称为下四分位数，表示全部观察值中有四分之一的数据取值比它小。
  - QU称为上四分位数，表示全部观察值中有四分之一的数据取值比它大。
  - IQR称为四分位数间距，是上四分位数QU与下四分位数QL之差，其间包含了全部观察值的一半。



❸检测与处理异常值

2. 箱线图分析

- 箱线图依据实际数据绘制，真实、直观地表现出了数据分布的本来面貌，且没有对数据做任何限制性要求（不像3σ原则要求数据服从正态分布），其判断异常值的标准以四分位数和四分位数间距为基础。
- 四分位数给出了数据分布的中心、散布和形状的某种指示，具有一定的鲁棒性，即25%的数据可以变得任意远而不会很大地扰动四分位数，所以异常值通常不能对这个标准施加影响。
- 鉴于此，箱线图识别异常值的结果比较客观，因此在识别异常值方面具有一定的优越性。

❸检测与处理异常值

2. 箱线图分析

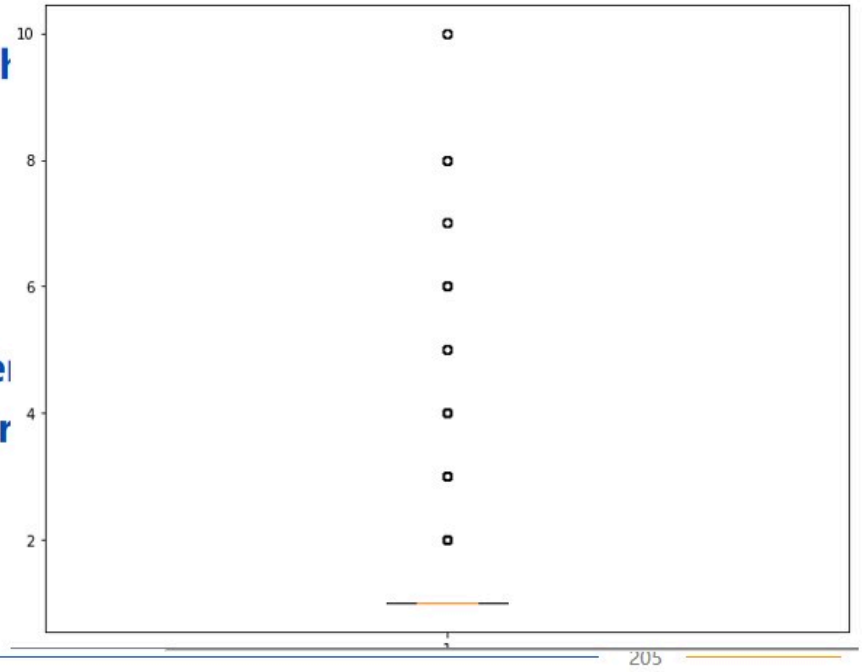
```
import matplotlib.pyplot as plt
plt.figure(figsize=(10,8))
p = plt.boxplot(detail['counts'].values, notch=True)        #画出箱线图
outlier1 = p['fliers'][0].get_ydata()                       #fliers为异常值的标签
plt.savefig('tmp/菜品异常数据识别.png')
plt.show()
print('销售量数据异常值个数为：',len(outlier1))
print('销售量数据异常值的最大值为：',max(outlier1))
print('销售量数据异常值的最小值为：',min(outlier1))
```

❸检测与处理异常值

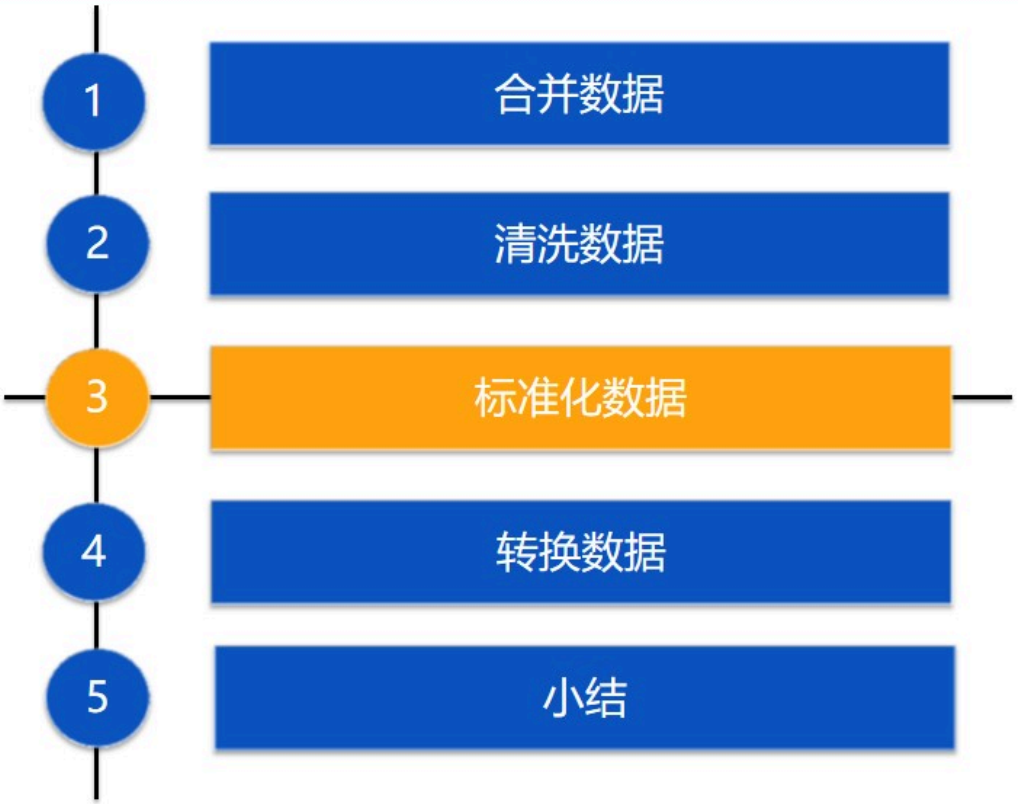
2.箱线图分析

```
import matplotlib.pyplot as plt
plt.figure(figsize=(10,8))
p = plt.boxplot(detail['counts'].values,notch
outlier1 = p['fliers'][0].get_ydata()
plt.savefig('tmp/菜品异常数据识别.png')
plt.show()
print('销售量数据异常值个数为：',len(outlier1))
print('销售量数据异常值的最大值为：',max(outlie
print('销售量数据异常值的最小值为：',min(outlier
```

```
销售量数据异常值个数为： 516
销售量数据异常值的最大值为： 10
销售量数据异常值的最小值为： 2
```



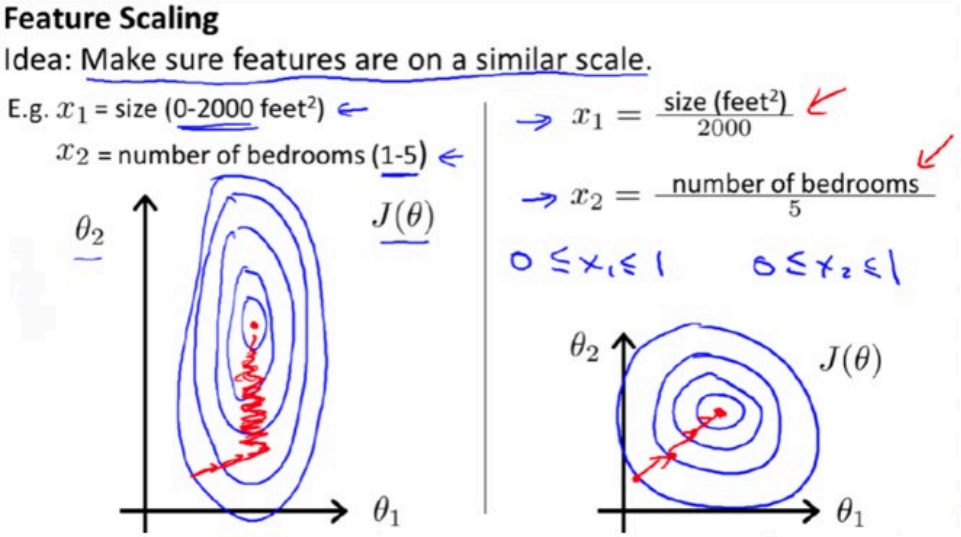
目录



标准化数据

为何进行数据标准化

- 比如Tiannic数据中我们拟合预测出缺失的年龄，这其中有几岁的小朋友，也有七八十的老人；房价预测时也有类似问题，有房屋尺寸和房屋数量两个特征，尺寸的值为0-2000平方英尺，而房间数量的值是0-5，以这两个数据作为横纵坐标，绘制损失函数的等高线图。



标准化数据

任务描述

- 不同特征之间往往具有不同的量纲，由此所造成的数值间的差异可能很大，在涉及空间距离计算或梯度下降法等情况时，不对其进行处理会影响到数据分析结果的准确性。为了消除特征之间量纲和取值范围差异可能造成的影响，需要对数据进行标准化处理，也可以称作规范化处理。

任务分析

（1）离差标准化数据

（2）标准差标准化数据

（3）小数定标标准化数据

雨课堂 Rain Classroom

❶离差标准化数据

离差标准化公式

- 离差标准化是对原始数据的一种线性变换，结果是将原始数据的数值**映射到[0,1]区间**，转换公式为：

$$X^* = \frac{X - \min}{\max - \min}$$

- 其中，max为样本数据的最大值，min为样本数据的最小值，max-min为极差。
- 离差标准化保留了原始数据值之间的联系，是消除量纲和数据取值范围影响最简单的方法。

❶离差标准化数据

示例：离差标准化订单详情表中的销量和售价

```
import pandas as pd
import numpy as np
detail = pd.read_csv('data/detail.csv', index_col=0, encoding = 'gbk')
def MinMaxScale(data):                    ## 自定义离差标准化函数
    data=(data-data.min())/(data.max()-data.min())
    return data
data1=MinMaxScale(detail['counts'])       #对菜品订单表销量做离差标准化
data2=MinMaxScale(detail['amounts'])      #对菜品订单表售价做离差标准化
data3=pd.concat([data1, data2], axis=1)
print('离差标准化之前销量和售价数据为：\n', detail[['counts','amounts']].head())
print('离差标准化之后销量和售价数据为：\n',data3.head())
```

❶离差标准化数据

示例：离差标准化订单详情表中的销量和售价

```
def MinMaxScale(data):                    ## 自定义离差标准化函数
    data=(data-data.min())/(data.max()-data.min())
    return data
data1=MinMaxScale(detail['counts'])       #对菜品订单表销量做离差标准化
data2=MinMaxScale(detail ['amounts'])     #对菜品订单表售价做离差标准化
data3=pd.concat([data1, data2], axis=1)
print('离差标准化之前销量和售价数据为：\n', detail[['counts','amounts']].head())
print('离差标准化之后销量和售价数据为：\n',data3.head())
```

离差标准化之前销量和售价数据为：

detail_id	counts	amounts
2956	1	49
2958	1	48
2961	1	30
2966	1	25
2968	1	13

离差标准化之后销量和售价数据为：

detail_id	counts	amounts
2956	0.0	0.271186
2958	0.0	0.265537
2961	0.0	0.163842
2966	0.0	0.135593
2968	0.0	0.067797

❶离差标准化数据

离差标准化的特点

- 通过代码运行，分析离差标准化前后的数据可以发现：
  - 数据的整体分布情况并不会随离差标准化而发生改变，原先取值较大的数据，在做完离差标准化后的值依旧较大。
  - 当数据和最小值相等的时候，通过离差标准化可以发现数据变为0。
  - 若数据极差过大就会出现，数据在离差标准化后数据之间的差值非常小的情况。
- 同时，还可以看出**离差标准化的缺点**：
  - 若数据集合中某个数值很大，则离差标准化的值就会接近于0，并且相互之间差别不大。若将来遇到超过目前属性[min,max]取值范围的时候，会引起系统出错，这时便需要重新确定min和max。

❷标准差标准化数据

标准差标准化的公式及特点

- 标准差标准化也叫零均值标准化或z分数标准化，是当前使用最广泛的数据标准化方法。经过该方法**处理的数据均值为0，标准差为1**，转化公式为：

$$X^* = \frac{X - \overline{X}}{\sigma}$$

- 其中$\overline{X}$为原始数据的均值，σ为原始数据的标准差

❷标准差标准化数据

示例：标准差标准化订单详情表中的销量和售价

```
def StandardScaler(data):                    ##自定义标准差标准化函数
    data=(data-data.mean())/data.std()
    return data
```

```
##对菜品订单表售价和销量做标准化
data4=StandardScaler(detail['counts'])
data5=StandardScaler(detail['amounts'])
data6=pd.concat([data4,data5],axis=1)
print('标准差标准化之前销量和售价数据为：\n',detail[['counts','amounts']].head())
print('标准差标准化之后销量和售价数据为：\n',data6.head())
```


雨课堂 Rain Classroom

❷标准差标准化数据

示例：标准差标准化订单详情表中的销量和售价

```
def StandardScaler(data):                    ##自定义标准差标准化函数
    data=(data-data.mean())/data.std()
    return data
data4=StandardScaler(detail['counts'])       ##对菜品订单表销量做标准化
data5=StandardScaler(detail['amounts'])      ##对菜品订单表售价做标准化
data6=pd.concat([data4,data5], axis=1)
print('标准差标准化之前销量和售价数据为：\n', detail[['counts','amounts']].head())
print('标准差标准化之后销量和售价数据为：\n', data6.head())
```

标准差标准化之前销量和售价数据为：

detail_id	counts	amounts
2956	1	49
2958	1	48
2961	1	30
2966	1	25
2968	1	13

标准差标准化之后销量和售价数据为：

detail_id	counts	amounts
2956	-0.177571	0.116671
2958	-0.177571	0.088751
2961	-0.177571	-0.413826
2966	-0.177571	-0.553431
2968	-0.177571	-0.888482

❸小数定标标准化数据

小数定标标准化公式及对比

- 通过移动数据的小数位数，**将数据映射到区间[-1,1]**，移动的小数位数取决于数据绝对值的最大值。

- 转化公式：$$X^* = \frac{X}{10^k}$$

雨课堂 Rain Classroom

❸小数定标标准化数据

示例：小数定标标准化订单详情表中的销量和售价

```
def DecimalScaler(data):                    ##自定义小数定标标准化函数
    data=data/10**np.ceil(np.log10(data.abs().max()))
    return data
```

```
##对菜品订单表售价和销量做标准化
data7=DecimalScaler(detail['counts'])
data8=DecimalScaler(detail['amounts'])
data9=pd.concat([data7,data8],axis=1)
print('小数定标标准化之前的销量和售价数据：\n',detail[['counts','amounts']].head())
print('小数定标标准化之后的销量和售价数据：\n',data9.head())
```

217

❸小数定标标准化数据

示例：小数定标标准化订单详情表中的销量和售价

```
def DecimalScaler(data):                    ##自定义小数定标标准化函数
    data=data/10**np.ceil(np.log10(data.abs().max()))
    return data
data7=DecimalScaler(detail['counts'])       ##对菜品订单表销量做标准化
data8=DecimalScaler(detail['amounts'])      ##对菜品订单表售价做标准化
data9=pd.concat([data7,data8],axis=1)
print('小数定标标准化之前的销量和售价数据：\n',detail[['counts','amounts']].head())
print('小数定标标准化之后的销量和售价数据：\n',data9.head())
```

小数定标标准化之前的销量和售价数据：

detail_id	counts	amounts
2956	1	49
2958	1	48
2961	1	30
2966	1	25
2968	1	13

小数定标标准化之后的销量和售价数据：

detail_id	counts	amounts
2956	0.1	0.049
2958	0.1	0.048
2961	0.1	0.030
2966	0.1	0.025
2968	0.1	0.013

标准化数据

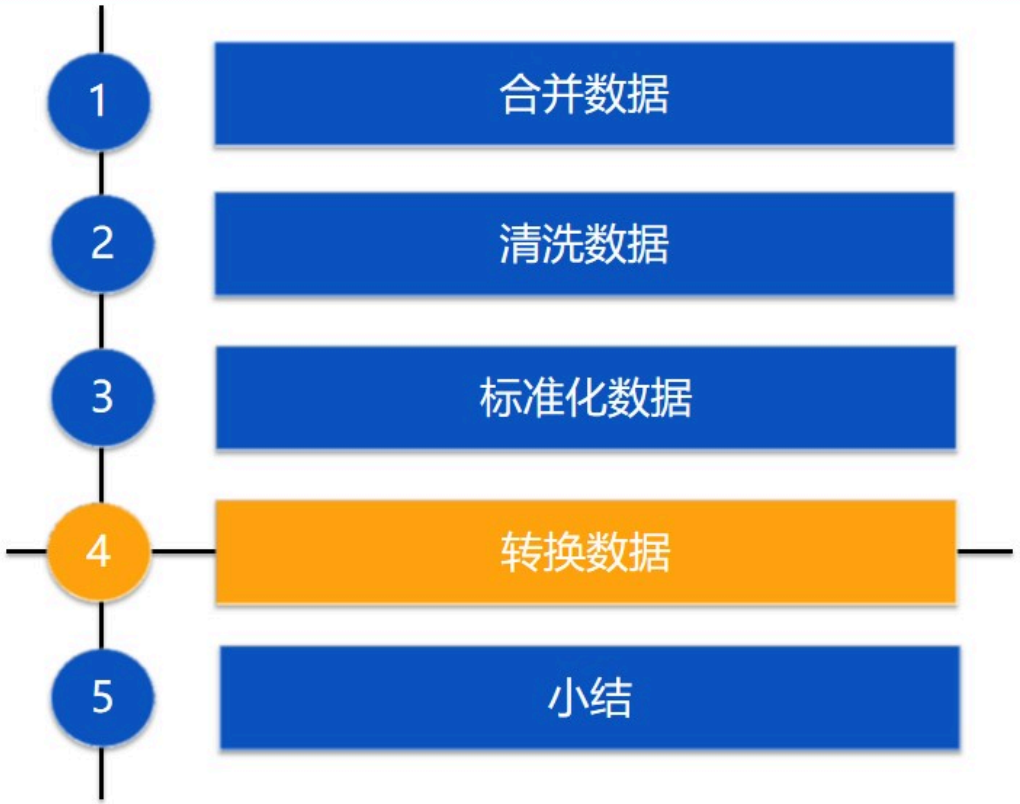
三种标准化方法对比

3种标准化方法各有其优势：

- 离差标准化方法简单，便于理解，标准化后的数据限定在[0,1]区间内。
- 标准差标准化受数据分布的影响最小。
- 小数定标标准化方法的适用范围广，并且受数据分布的影响小，相比较前两种方法，该方法使用程度适中。

雨课堂 Rain Classroom

目录



转换数据

任务描述

- 数据分析的预处理工作除了数据清理、数据合并以及数据标准化之外，还包括数据变换的过程。
- 数据即使经过了清洗、合并和标准化，依旧不能直接拿来做分析建模。
- 为了能够将数据分析工作继续往前推进，需要对数据做一些合理的转换，使之符合分析的要求。

任务分析

（1）**类别型数据**变成数值型数据（哑变量处理）

（2）**离散化连续型数据**（等宽法、等频法、聚类分析法）

❶哑变量处理类别数据

哑变量处理

- 数据分析模型中有相当一部分的算法模型都要求输入的特征为数值型，但实际数据中特征的类型不一定只有数值型，还会存在相当一部分的类别型，这部分的特征需要经过哑变量处理才可以放入模型之中。
- 哑变量处理的原理示例如图。

哑变量处理前

	城市
1	广州
2	上海
3	杭州
4	北京
5	深圳
6	北京
7	上海
8	杭州
9	广州
10	深圳

哑变量处理后

	城市_广州	城市_上海	城市_杭州	城市_北京	城市_深圳
1	1	0	0	0	0
2	0	1	0	0	0
3	0	0	1	0	0
4	0	0	0	1	0
5	0	0	0	0	1
6	0	0	0	1	0
7	0	1	0	0	0
8	0	0	1	0	0
9	1	0	0	0	0
10	0	0	0	0	1

雨课堂 Rain Classroom

❶哑变量处理类别数据

哑变量处理

- 哑变量(Dummy Variable)：也叫虚拟变量。引入哑变量的目的是将不能够定量处理的变量量化。
- 如职业、性别对收入的影响，战争、自然灾害对GDP的影响，季节对某些产品(如冷饮)销售的影响等等。
- 这种“量化”通常是通过引入“哑变量”来完成的。根据这些因素的属性类型，构造只取“0”或“1”的人工变量，通常称为哑变量，记为D。

❶哑变量处理类别数据

哑变量处理 —— 实例

- 假设变量“职业”的取值分别为：工人、农民、学生、企业职员、其他，5种选项。
- 我们可以增加4个哑变量来代替“职业”这个变量：
  - D1(1=工人/0=非工人)
  - D2(1=农民/0=非农民)
  - D3(1=学生/0=非学生)
  - D4(1=企业职员/0=非企业职员)
- 最后一个选项“其他”的信息已经包含在这4个变量中了（D1/D2/D3/D4 = 0，则就是其他职业），所以不需要再增加一个D5了。
- 这个过程就是引入哑变量的过程。
- 其实在联合(结合)分析(conjoint analysis)中，就是利用哑变量来分析各个属性的效用值的。

❶哑变量处理类别数据

get_dummies函数

- Python中可以利用pandas库中的get_dummies函数对类别型特征进行哑变量处理。

pandas.get_dummies(data, prefix=None, prefix_sep='_', dummy_na=False, columns=None, sparse=False, drop_first=False)

参数名称	说明
data	接收array、DataFrame或者Series。表示需要哑变量处理的数据。无默认。
prefix	接收string、string的列表或者string的dict。表示哑变量化后列名的前缀。默认为None。
prefix_sep	接收string。表示前缀的连接符。默认为‘_’。
dummy_na	接收boolean。表示是否为Nan值添加一列。默认为False。
columns	接收类似list的数据。表示DataFrame中需要编码的列名。默认为None，表示对所有object和category类型进行编码。
sparse	接收boolean。表示虚拟列是否是稀疏的。默认为False。
drop_first	接收boolean。表示是否通过从k个分类级别中删除第一级来获得k-1个分类级别。默认为False。

❶哑变量处理类别数据

get_dummies函数 —— 应用示例

- 菜品订单详情表中的菜品名称就是类别型特征，利用get_dummies函数对其进行哑变量处理。

```
import pandas as pd
import numpy as np
detail = pd.read_csv('data/detail.csv',encoding = 'gbk')
data=detail.loc[0:5,'dishes_name']          #抽取部分数据做演示
print('哑变量处理前的数据为：\n', data)
print('哑变量处理后的数据为：\n', pd.get_dummies(data))
```

```
哑变量处理前的数据为：
 0      蒜蓉生蚝
1     蒙古烤羊腿
2      大蒜苋菜
3     芝麻烤紫菜
4      蒜香包
5      白斩鸡
Name: dishes_name, dtype: object
```

```
哑变量处理后的数据为：
   大蒜苋菜  白斩鸡  芝麻烤紫菜  蒙古烤羊腿  蒜蓉生蚝  蒜香包
0     0     0      0      1     0
1     0     0      0      0     1
2     1     0      0      0     0
3     0     0      1      0     0
4     0     0      0      0     0
5     0     1      0      0     0
```

❶哑变量处理类别数据

哑变量处理的特点

- 从代码运行结果看出，对于一个类别型特征，若其取值有m个，则经过哑变量处理后就变成了m个二元特征，并且这些特征互斥，每次只有一个激活，这使得数据变得稀疏。

总之：

- 对类别型特征进行哑变量处理主要解决了部分算法模型无法处理类别型数据的问题，这在一定程度上起到了扩充特征的作用。
- 由于数据变成了稀疏矩阵的形式，因此也加速了算法模型的运算速度。

❷离散化连续型数据

离散化

- **(Why)**某些模型算法，特别是某些分类算法如决策树算法和关联分析算法等，要求数据是离散的，此时就需要将连续型特征（数值型）变换成离散型特征（类别型）。
- **(How)**连续特征的离散化就是先设定若干个离散的划分点，进而将取值范围划分为一些离散化的区间，最后用不同的符号或整数值代表落在每个子区间中的数据值。
- 因此，**离散化涉及两个子任务**，即确定分类数以及如何将连续型数据映射到这些类别型数据上。其原理如右图。
- **常用的离散化方法主要有3种：等宽法、等频法、聚类分析法**

离散化处理前

	年龄
1	18
2	23
3	35
4	54
5	42
6	21
7	60
8	63
9	41
10	38

离散化处理后

	年龄
1	(17.955，27]
2	(17.955，27]
3	(27，36]
4	(45，54]
5	(36，45]
6	(17.955，27]
7	(54，63]
8	(54，63]
9	(36，45]
10	(36，45]

❷离散化连续型数据

1. 等宽法

- 将数据的值域分成具有相同宽度的区间，区间的个数由数据本身的特点决定或者用户指定。
- pandas提供了cut函数，可以进行连续型数据的等宽离散化，其基础语法格式如下：

pandas.cut(x, bins, right=True, labels=None, retbins=False, precision=3, include_lowest=False)

参数名称	说明
x	接收数组或Series。代表需要进行离散化处理的数据。无默认。
bins	接收int，list/array/tuple。若为int，代表离散化后的类别数目；若为序列类型的数据，则表示进行切分的区间，每两个数间隔为一个区间。无默认。
right	接收boolean。代表右侧是否为闭区间。默认为True。
labels	接收list，array。代表离散化后各个类别的名称。默认为空。
retbins	接收boolean。代表是否返回区间标签。默认为False。
precision	接收int。显示的标签的精度。默认为3。

❷离散化连续型数据

1. 等宽法离散化示例

- 菜品售价amount使用cut函数进行等宽法离散化处理

```
price = pd.cut(detail['amounts'],5)
print('离散化后5条记录售价分布为：\n' ,price.value_counts())
```

```
离散化后5条记录售价分布为：
 (0.823, 36.4]     5461
(36.4, 71.8]      3157
(71.8, 107.2]      839
(142.6, 178.0]     426
(107.2, 142.6]     154
Name: amounts, dtype: int64
```

❷离散化连续型数据

1. 等宽法

- 使用等宽法离散化的**缺陷**为：等宽法离散化对数据分布具有较高要求
  - 若数据分布不均匀，那么各个类的数目也会变得非常不均匀，有些区间包含许多数据，而另外一些区间的数据极少，这会严重损坏所建立的模型。

❷离散化连续型数据

2. 等频法--示例

.quantile：K分位函数，返回DataFrame的index就是k，value就是分位值。

- cut函数虽然不能直接实现等频离散化，但是可以通过定义将相同数量的记录放进每个区间。

```
def SameRateCut(data, k):
    w=data.quantile(np.arange(0, 1+1.0/k, 1.0/k))
    data=pd.cut(data,w)
    return data
result=SameRateCut(detail['amounts'], 5).value_counts()
print('菜品数据等频法离散化后各个类别数目分布状况为：\n', result)
```

```
In [18]: w
Out[18]:
0.0      1.0
0.2     18.0
0.4     32.0
0.6     39.0
0.8     58.0
1.0    178.0
```

```
菜品数据等频法离散化后各个类别数目分布状况为：
 (18.0, 32.0]     2107
(39.0, 58.0]     2080
(32.0, 39.0]     1910
(1.0, 18.0]      1891
(58.0, 178.0]    1863
Name: amounts, dtype: int64
```

.cut：参数如果是array，也可以是区间切分，不一定是等分，这里就实现了等频分割。

❷离散化连续型数据

2. 等频法

- 等频法比较于等宽法而言，避免了类分布不均匀的问题
- 但却有可能将数值非常接近的两个值分到不同的区间以满足每个区间中固定的数据个数。

❷离散化连续型数据

3. 基于聚类分析的方法

- 一维聚类的方法包括两个步骤：
  - 首先，将连续型数据用聚类算法（如K-Means算法等）进行聚类。
  - 然后，处理聚类得到的簇，将合并到一个簇的连续型数据做同一标记。

- 聚类分析的离散化方法需要用户指定簇的个数，用来决定产生的区间数。

- 在下一章“机器学习”中再详细介绍

❷离散化连续型数据

3. 基于聚类分析的离散化方法示例

```
from sklearn.cluster import KMeans                          #引入KMeans机器学习算法
def KmeanCut(data,k):                                       #自定义数据离散化函数
    kmodel=KMeans(n_clusters=k, n_jobs=4)                       #建立模型，n_jobs是并行数
    kmodel.fit(data.values.reshape((len(data), 1)))             #训练模型
    c=pd.DataFrame(kmodel.cluster_centers_).sort_values(0)      #输出聚类中心并排序
    w=c.rolling(2).mean.iloc[1:]                                #相邻两项求中点，作为边界点
    w=[0]+list(w[0])+[data.max()]                               #把首末边界点加上
    data=pd.cut(data,w)
    return data
result=KmeanCut(detail['amounts'],5).value_counts()         #菜品售价聚类分析方法的离散化
print('菜品售价聚类离散化后各个类别数目分布状况为：\n', result)
```

❷离散化连续型数据

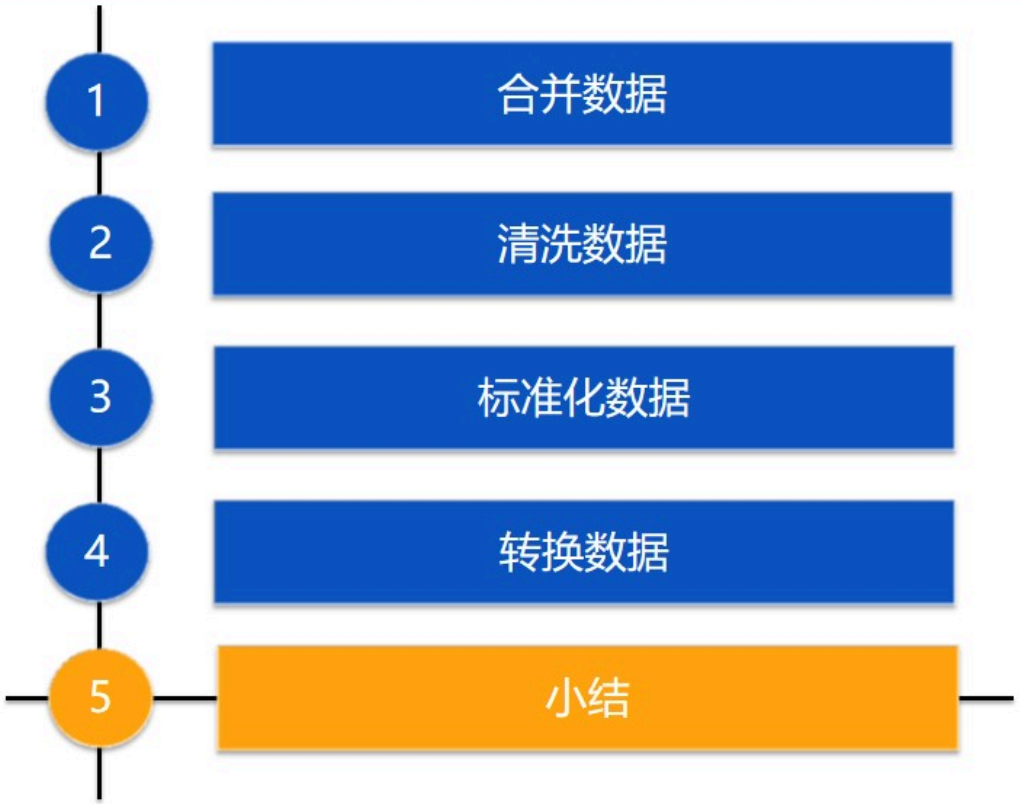
3. 基于聚类分析的离散化方法示例

```
菜品售价聚类离散化后各个类别数目分布状况为：
 (22.31, 43.51]       3690
(43.51, 73.945]       2474
(0.0, 22.31]          2454
(73.945, 131.858]      993
(131.858, 178.0]       426
Name: amounts, dtype: int64
```

- k-Means聚类分析的离散化方法可以很好地根据现有特征的数据分布状况进行聚类，但是由于k-Means算法本身的缺陷，用该方法进行离散化时依旧需要指定离散化后类别的数目。此时需要配合聚类算法评价方法，找出最优的聚类簇数目。

雨课堂 Rain Classroom

目录



小结

本章以菜品数据为例子，实现了数据分析的数据预处理过程，即数据清洗、数据合并、数据标准化和数据转换。这四个步骤并不存在严格的先后关系，实际工作中往往需要交叉工作。

- **数据清洗**主要介绍了对重复数据、缺失值和异常值的处理。
  - 重复数据处理细分为记录去重和特征去重。
  - 缺失值处理方法分为删除、替换和插值。
  - 异常值介绍了3σ原则和箱线图识别这两种识别方法。
- **数据合并**是将多个数据源中的数据合并存放到一个数据存储的过程。
- **数据标准化**介绍了如何将不同量纲的数据转化为可以相互比较的标准化数据。
- **数据转换**介绍了如何从不同的应用角度对已有特征进行转换。

Thank you!